

BCSE205L
Computer Architecture and Organization

Module 1 – Introduction to CAO

Dr. P.Keerthika

Associate Professor

School of Computer Science and Engineering
Vellore Institute of Technology, Vellore

BCSE205L - Computer Architecture and Organization

• Course Objectives

1. To acquaint students with the basic concepts of fundamental component, architecture, register organization and performance metrics of a computer and to impart the knowledge of data representation in binary and to understand the implementation of arithmetic algorithms in a typical computer.
2. To teach students how to describe machine capabilities and design an effective data path design for instruction execution. To introduce students to syntax and semantics of machine level programming.
3. To make students understand the importance of memory systems, IO interfacing techniques and external storage and their performance metrics for a typical computer. And explore various alternate techniques for improving the performance of a processor.

BCSE205L - Computer Architecture and Organization

• Course Outcomes

On completion of this course, student should be able to:

1. Differentiate Von Neumann, Harvard, and CISC and RISC architectures. Analyze the performance of machine with different capabilities. Recognize different instruction formats and addressing modes. Validate efficient algorithm for fixed point and floating point arithmetic operations.
2. Explain the importance of hierarchical memory organization. Able to construct larger memories. Analyze and suggest efficient cache mapping technique and replacement algorithms for given design requirements. Demonstrate hamming code for error detection and correction.
3. Understand the need for an interface. Compare and contrast memory mapping and IO mapping techniques. Describe and Differentiate different modes of data transfer. Appraise the synchronous and asynchronous bus for performance and arbitration.
4. Assess the performance of IO and external storage systems. Classify parallel machine models. Analyze the pipeline hazards and solutions.

BCSE205L - Computer Architecture and Organization

- Syllabus

Module:1	Introduction To Computer Architecture and Organization	5 Hours
Overview of Organization and Architecture –Functional components of a computer: Registers and register files - Interconnection of components - Overview of IAS computer function - Organization of the von Neumann machine - Harvard architecture - CISC & RISC Architectures.		
Module:2	Data Representation and Computer Arithmetic	5 Hours
Algorithms for fixed point arithmetic operations: Multiplication (Booths, Modified Booths), Division (restoring and non-restoring) - Algorithms for floating point arithmetic operations - Representation of nonnumeric data (character codes).		
Module:3	Instruction Sets and Control Unit	9 Hours
Computer Instructions: Instruction sets, Instruction Set Architecture, Instruction formats, Instruction set categories - Addressing modes - Phases of instruction cycle – ALU - Data-path and control unit: Hardwired control unit and Micro programmed control unit - Performance metrics: Execution time calculation, MIPS, MFLOPS.		

BCSE205L - Computer Architecture and Organization

- Syllabus

Module:4	Memory System Organization and Architecture	7 Hours
Memory systems hierarchy: Characteristics, Byte Storage methods, Conceptual view of memory cell - Design of scalable memory using RAM's- ROM's chips - Construction of larger size memories - Memory Interleaving - Memory interface address map- Cache memory: principles, Cache memory management techniques, Types of caches, caches misses, Mean		
Module:5	Interfacing and Communication	5 Hours
I/O fundamentals: handshaking, buffering, I/O Modules - I/O techniques: Programmed I/O, Interrupt-driven I/O, Direct Memory Access, Direct Cache Access - Interrupt structures: Vectored and Prioritized-interrupt overhead - Buses: Synchronous and asynchronous - Arbitration.		
Module:6	Subsystems	5 Hours
External storage systems: Solid state drivers - Organization and Structure of disk drives: Electronic- magnetic and optical technologies - Reliability of memory systems - Error detecting and error correcting systems - RAID Levels - I/O Performance		

BCSE205L - Computer Architecture and Organization

- Syllabus

Module:7	High Performance Processors	7 Hours
Classification of models - Flynn's taxonomy of parallel machine models (SISD, SIMD, MISD, MIMD) - Pipelining: Two stages, Multi stage pipelining, Basic performance issues in pipelining, Hazards, Methods to prevent and resolve hazards and their drawbacks - Approaches to deal branches - Superscalar architecture: Limitations of scalar pipelines, superscalar versus super pipeline architecture, superscalar techniques, performance evaluation of superscalar architecture - performance evaluation of parallel processors: Amdahl's law, speed-up and efficiency.		
Module:8	Contemporary Issues	2 Hours

BCSE205L - Computer Architecture and Organization

- Text and Reference Books

Text Book(s)	
1	David A. Patterson and John L. Hennessy, Computer Organization and Design -The Hardware / Software Interface 6 th Edition, Morgan Kaufmann, 2020
Reference Book(s)	
1	Computer Architecture and Organization-Designing for Performance, William Stallings, Tenth edition, Pearson Education series, 2016
2	Carl Hamacher, Zvonko Vranesic, Safwat Zaky, Computer organization, Mc Graw Hill, Fifth edition, Reprint 2011.

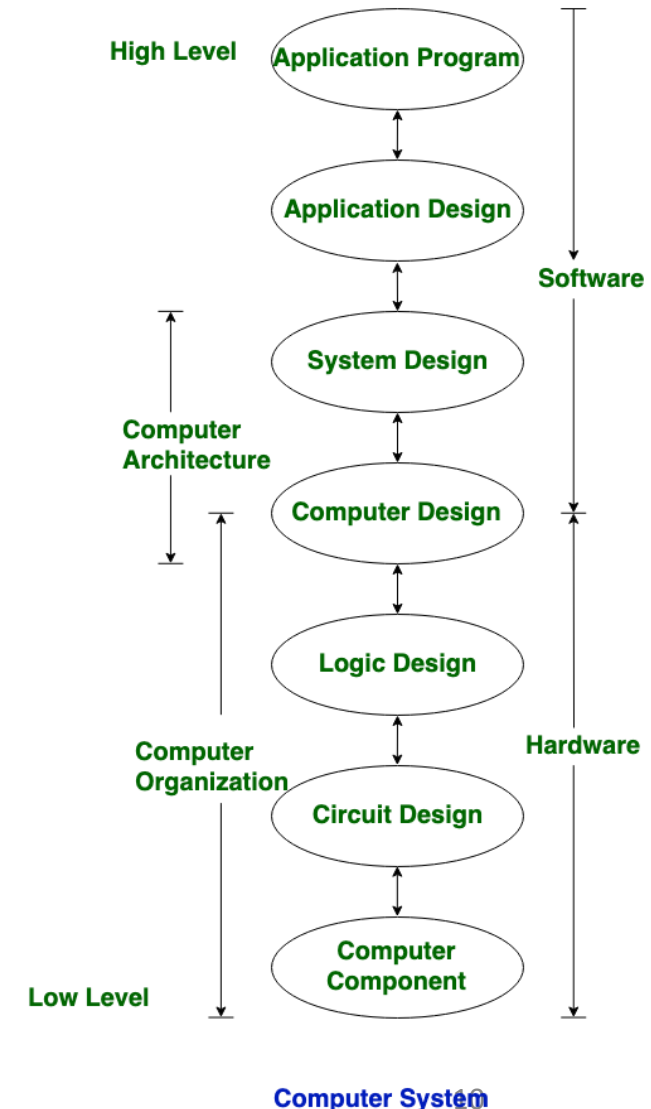
Overview of Organization and Architecture

- Computer Architecture and Computer Organization are **related** but **distinct concepts** in the field of computer science.
- **Computer Architecture**
 - refers to the **design of the internal workings** of a computer system, including the CPU, memory, and other hardware components.
 - involves decisions about the **organization of the hardware**, such as the instruction set architecture, the data path design, and the control unit design.
 - is concerned with **optimizing the performance** of a computer system and ensuring that it can execute instructions **quickly and efficiently**.

Overview of Organization and Architecture

• Computer Architecture

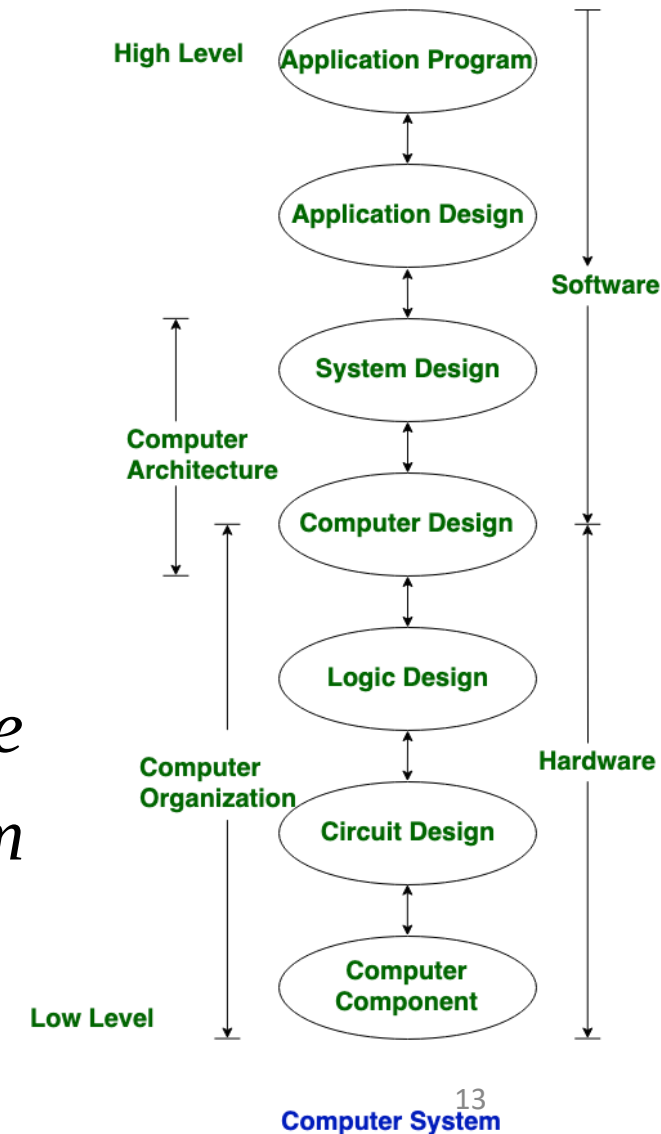
- refers to those **attributes of a system** visible to a programmer.
- **Architectural attributes** includes
 - Instruction set
 - the number of bits used to represent various data types (e.g., numbers, characters),
 - I/O mechanisms and techniques
 - Addressing memory
- Example: “Whether a computer will have a **multiply instruction?**”
 - **Architectural design issue.**



Overview of Organization and Architecture

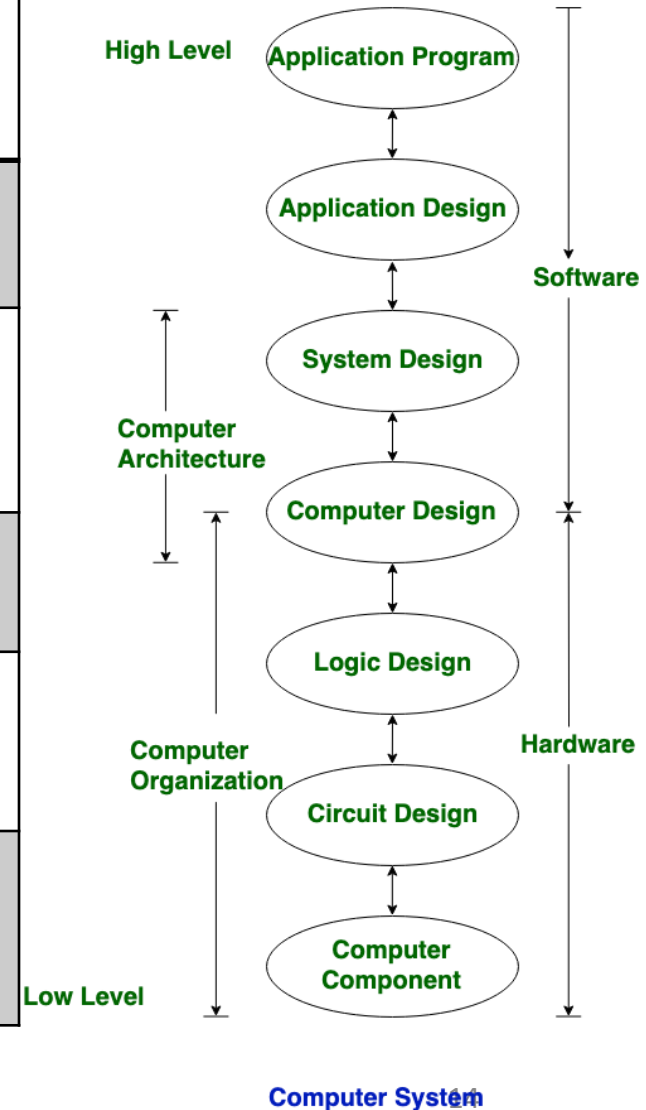
- **Organizational attributes** includes
 - Hardware details transparent to the programmer
 - Control signals
 - Interfaces between the computer and peripherals
 - Memory technology used
- Example: *“Whether that multiply instruction will be implemented by a special **multiply unit** or by a mechanism that makes repeated use of the **add unit** of the system”*

- **Organizational issue**



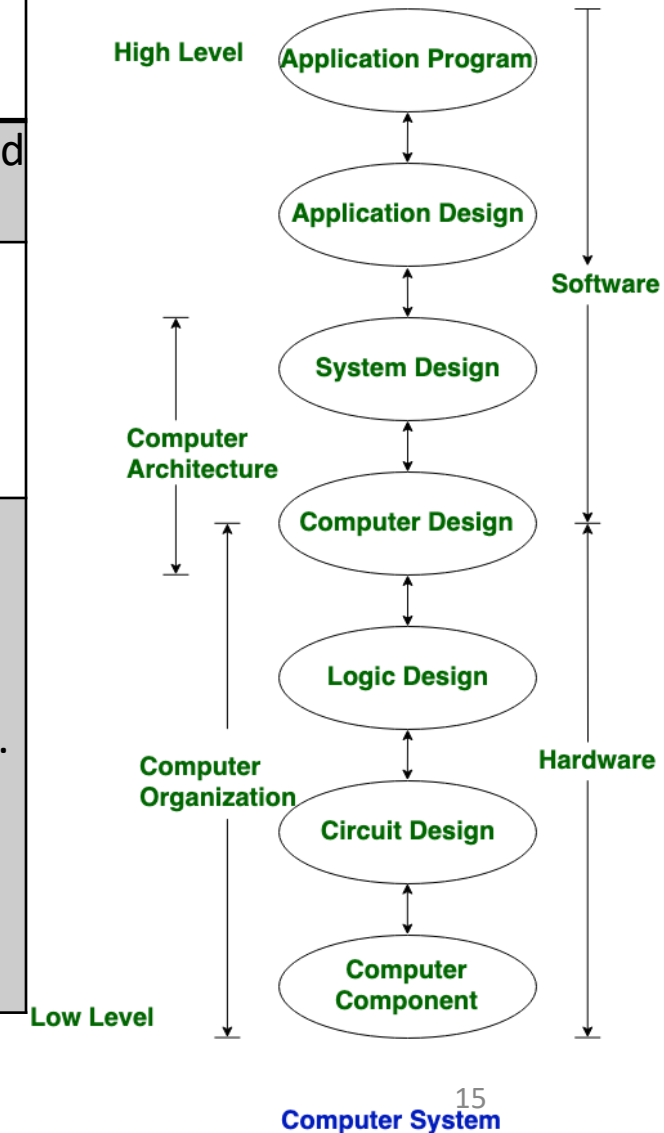
Overview of Organization and Architecture

	Computer Architecture	Computer Organization
1.	Architecture describes what the computer does	Organization describes how it does it.
2.	Computer Architecture deals with functional behavior of a computer system.	Computer Organization deals with structural relationship.
3.	it deals with high-level design issues.	it deals with low-level design issues.
4.	As a programmer, you can view architecture as a series of instructions, addressing modes, and registers.	The implementation of the architecture is called organization.
5.	For designing a computer, its architecture is fixed first.	For designing a computer, an organization is decided after its architecture.



Overview of Organization and Architecture

	Computer Architecture	Computer Organization
6.	Computer Architecture is also called as Instruction Set Architecture (ISA) .	Computer Organization is frequently called as micro architecture .
7.	Computer Architecture comprises logical functions such as instruction sets, registers, data types and addressing modes .	Computer Organization consists of physical units like circuit designs, peripherals and adders .
8.	The different architectural categories found in our computer systems are: • Von-Neumann Architecture • Harvard Architecture • Instruction Set Architecture • Micro-architecture • System Design	CPU organization is classified into three categories based on the number of address fields: • Organization of a single Accumulator. • Organization of general registers • Stack organization

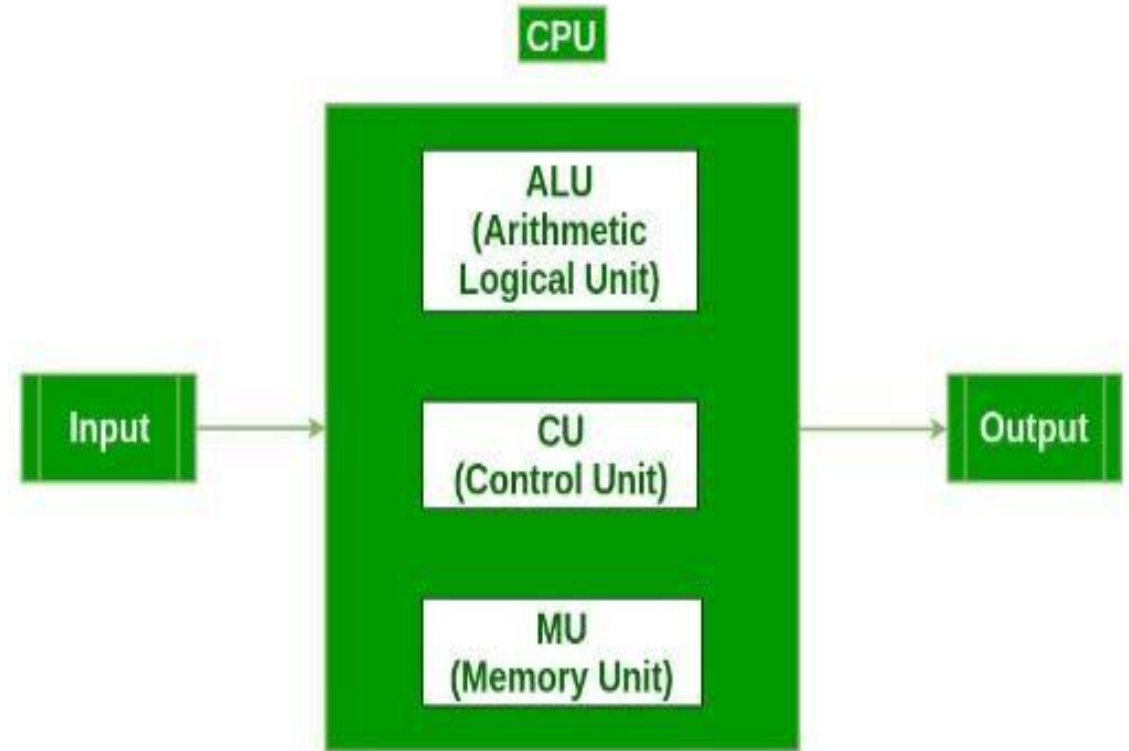


Overview of Organization and Architecture

- **Organizational decision** may be based on
 - the anticipated frequency of use of the multiply instruction
 - the relative speed of the two approaches
 - cost and physical size of a special multiply unit
- Many computer manufacturers offer a **family of computer models**, all with the **same architecture** but with **differences in organization**.
 - Architecture – Long term (span for many years)
 - Organization – Short term (changing with technology)
- **Example** - IBM System/370 architecture (1970)
 - Customer with modest requirements – prefer Cheaper and slower model
 - Customer with highest requirements – prefer Expensive and faster model

Functional Components of Computer

- Input Unit
- Output Unit
- Memory Unit
- Central Processing Unit (CPU)
 - Arithmetic and Logic Unit (ALU)
 - Control Unit
 - Memory Registers



Functional components of a computer

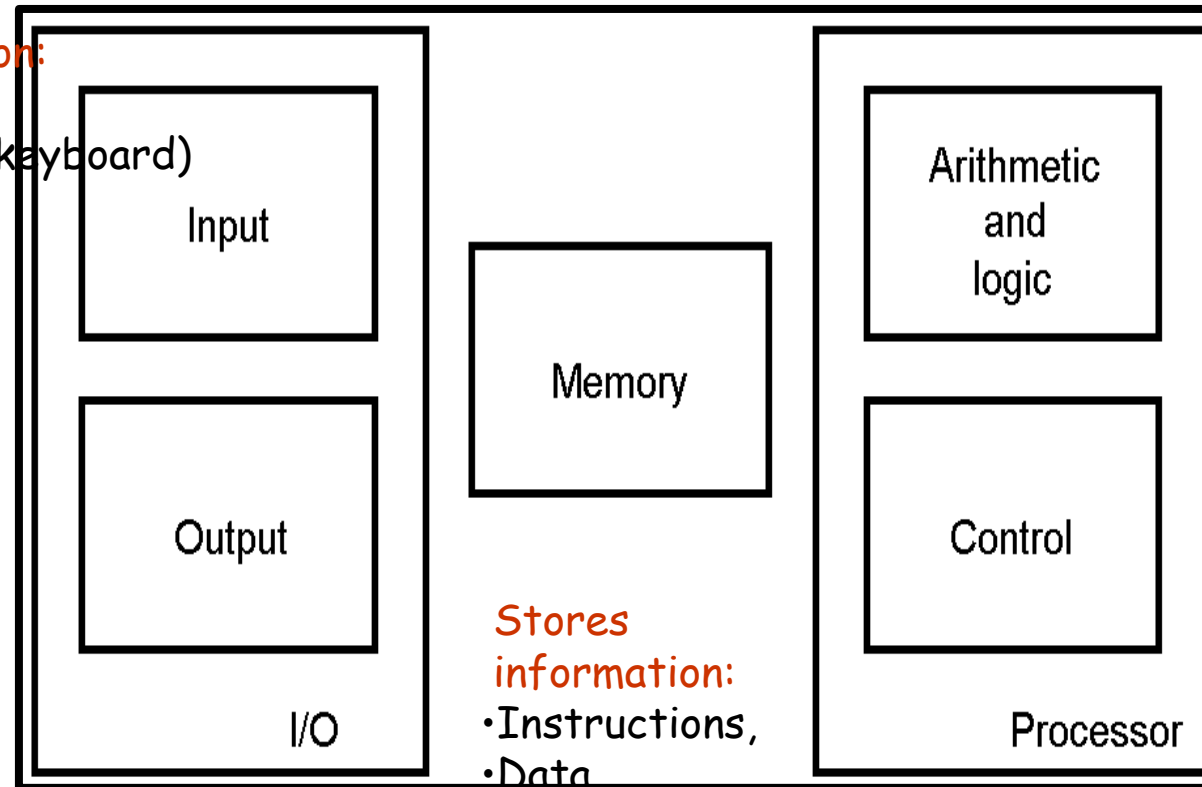
- **Basic functional units of a computer** – Five functionally independent main parts.

Input unit accepts information:

- Human operators,
- Electromechanical devices (keyboard)
- Other computers

Output unit sends results of processing:

- To a monitor display,
- To a printer



Arithmetic and logic unit(ALU):

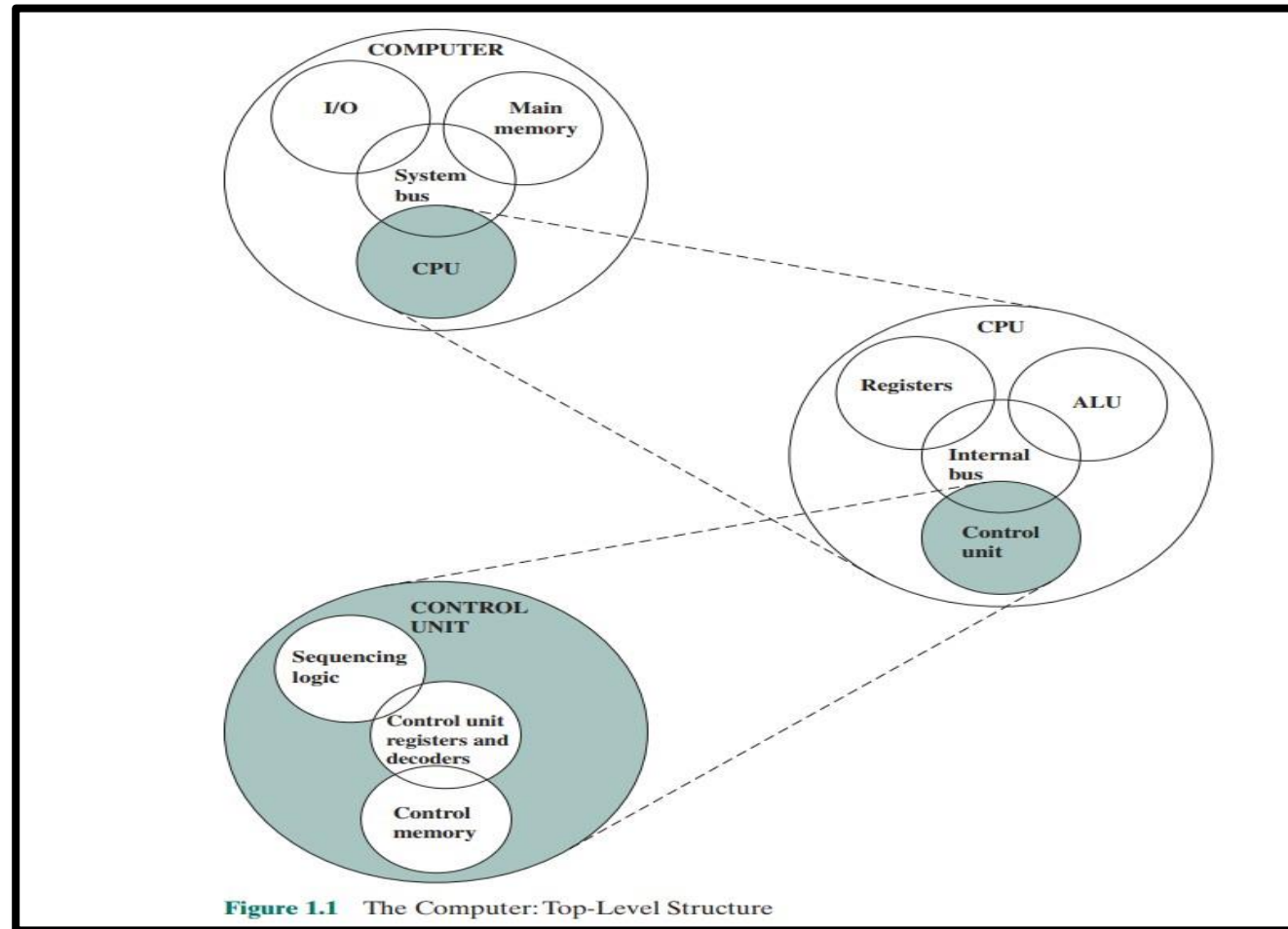
- Performs the desired operations on the input information as determined by instructions in the memory

Control unit coordinates various actions

- Input,
- Output
- Processing

Functional components of a computer

- **Basic functional units of a computer** – Five functionally independent main parts.



Functional components of a computer

- **Input Unit :**
 - A computer accepts the information through input device.
 - Input device writes data into the memory through the data path. Some of the common input devices are keyboard, mouse, joystick, scanner etc.
- **Output Unit:**
 - The Output unit send the processed result to the user using output device.
 - It reads the data from memory through data path.
 - Some devices are monitor, printer etc.

Functional components of a computer

- **Central Processing Unit (CPU) :**
 - Brain of the computer - control center of the computer.
 - It first fetches instructions from memory and then interprets them so as to know what is to be done.
 - CPU executes or performs the required computation and then either stores the output or displays on the output device
- **Arithmetic and Logic Unit (ALU) :**
 - performs mathematical calculations and takes logical decisions. Arithmetic calculations include addition, subtraction, multiplication and division

Functional components of a computer

- Central Processing Unit (CPU) :
 - Control Unit :
 - coordinates and controls the data flow in and out of CPU and also controls all the operations of ALU, memory registers and also input/output units.
 - It is also responsible for carrying out all the instructions stored in the program.
 - It decodes the fetched instruction, interprets it and sends control signals to input/output devices until the required operation is done properly by ALU and memory.

Functional components of a computer

- Central Processing Unit (CPU) :
 - Memory Registers :
 - A register is a temporary unit of memory in the CPU. These are used to store the data which is directly used by the processor.
 - Registers can be of different sizes(16 bit, 32 bit, 64 bit and so on) and each register inside the CPU has a specific function like storing data, storing an instruction, storing address of a location in memory etc.
 - The user registers can be used by an assembly language programmer for storing operands, intermediate results

Functional components of a computer

- Memory Unit (MU) :
 - Memory unit stores instructions and data.
 - Recall, data is represented as a series of bits.
 - To store data, memory unit thus stores bits.
 - Processor reads instructions and reads/writes data from/to the memory during the execution of a program.
 - In theory, instructions and data could be fetched one bit at a time.
 - In practice, a group of bits is fetched at a time.
 - Group of bits stored or retrieved at a time is termed as “word”
 - Number of bits in a word is termed as the “word length” of a computer.
 - In order to read/write to and from memory, a processor should know where to look:
 - “Address” is associated with each word location.

Functional components of a computer

- Memory Unit (MU) :
 - Processor reads/writes to/from memory based on the memory address:
 - Access any word location in a short and fixed amount of time based on the address.
 - Random Access Memory (RAM) provides fixed access time independent of the location of the word.
 - Access time is known as “Memory Access Time”.
 - Memory and processor have to “communicate” with each other in order to read/write information.
 - In order to reduce “communication time”, a small amount of RAM (known as Cache) is tightly coupled with the processor.
 - Modern computers have three to four levels of RAM units with different speeds and sizes:
 - Fastest, smallest known as Cache
 - Slowest, largest known as Main memory.

Memory unit (contd..)

- Primary storage is insufficient to store large amounts of data and programs.
 - Primary storage can be added, but it is expensive.
- Store large amounts of data on secondary storage devices:
 - Magnetic disks and tapes,
 - Optical disks (CD-ROMS).
 - Access to the data stored in secondary storage is slower, but take advantage of the fact that some information may be accessed infrequently.
- Cost of a memory unit depends on its access time, lesser access time implies higher cost.

Information in a computer -- *Instructions*

- Instructions specify commands to:
 - Transfer information within a computer (e.g., from memory to ALU)
 - Transfer of information between the computer and I/O devices (e.g., from keyboard to computer, or computer to printer)
 - Perform arithmetic and logic operations (e.g., Add two numbers, Perform a logical AND).

Information in a computer -- *Instructions*

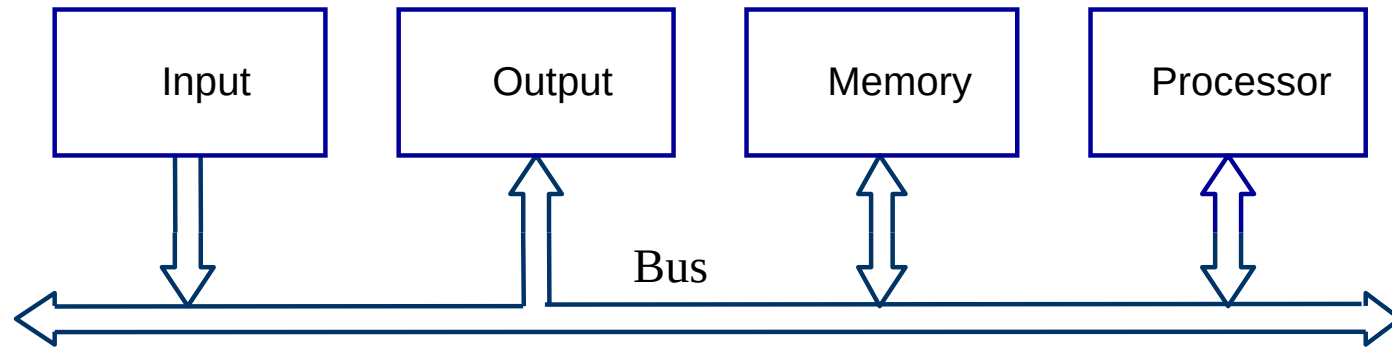
- A sequence of instructions to perform a task is called a **program**, which is stored in the memory.
- **Processor fetches instructions** that make up a program from the memory and **performs** the **operations** stated in those instructions.
- **What do the instructions operate upon?**
-----Data

Information in a computer -- *Data*

- **Data** are the “**operands**” upon which instructions operate.
- Data could be:
 - Numbers,
 - Encoded characters.
- Data, in a broad sense means any digital information.
- Computers use data that is encoded as a string of binary digits called bits.

How are the functional units connected?

- For a computer to achieve its operation, the functional units need to communicate with each other.
- In order to communicate, they need to be connected.



- Functional units may be connected by a group of parallel wires.
- The group of parallel wires is called a bus.
- Each wire in a bus can transfer one bit of information.
- The number of parallel wires in a bus is equal to the word length of a computer

Drawbacks of the Single Bus Structure

- The devices connected to a bus **vary widely in their speed of operation**
 - Some devices are relatively slow, such as printer and keyboard
 - Some devices are considerably fast, such as optical disks
 - Memory and processor units operate are the fastest parts of a computer
- Efficient transfer mechanism thus is needed to cope with this problem
 - A common approach is to **include buffer registers** with the devices to hold the information during transfers
 - An another approach is to use **two-bus structure** and an additional transfer mechanism
 - A high-performance bus, a low-performance, and a bridge for transferring the data between the two buses - ARMA Bus belongs to this structure.

Basic Operational Concepts

- **Activity** in a computer is governed by **instructions**.
- To perform a task, an **appropriate program** consisting of a list of instructions is **stored in the memory**.
- **Individual instructions** are brought from the **memory into the processor**, which executes the specified operations.
- Data to be used as operands are also stored in the memory.
- Typical Instruction - **Add LOCA, R0**
- What is the function of **Add LOCA, R0** and How does it execute!!??

Basic Operational Concepts

- Typical Instruction - **Add LOCA, R0**

- Add the operand at memory location LOCA to the operand in a register R0 in the processor.
- Place the sum into register R0.
- The original contents of LOCA are preserved.
- The original contents of R0 is overwritten.

The instruction combines a memory access operation with an ALU operation.

- Steps:

1. Instruction is fetched from the memory into the processor.
2. The operand at LOCA is fetched and added to the contents of R0
3. The resulting sum is stored in register R0.

Basic Operational Concepts

- **Separate Memory Access and ALU Operation**

- In many modern computers, those two types of operation are performed by separate instructions – Performance reason.
- Same instruction can be realized by the following two instruction sequence.

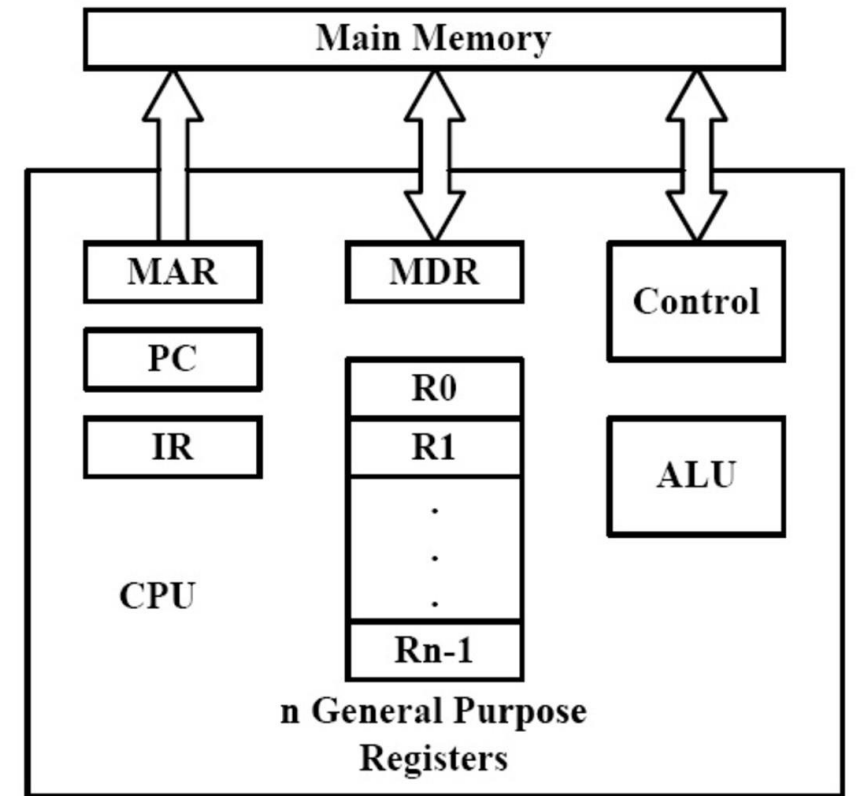
Load LOCA, R1 # Transfer content of LOCA to R1

Add R1, R0 # $R0 = R1 + R0$

- Here the content of R1 and R0 destroyed & LOCA content is preserved.

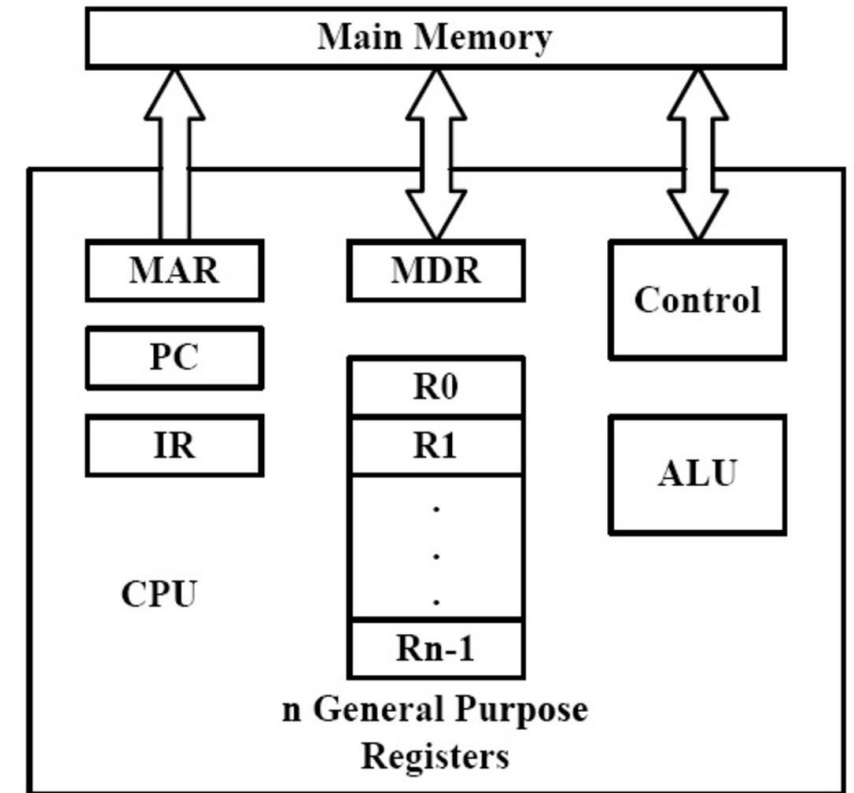
Basic Operational Concepts

- **Connection between the processor and the memory**
 - Instruction register (IR)
 - Program counter (PC)
 - Memory address register (MAR)
 - Memory data register (MDR)
 - **General-purpose register ($R_0 - R_{n-1}$)** are used to **store temporary data** during any ongoing operation. Its content can be accessed by assembly programming.



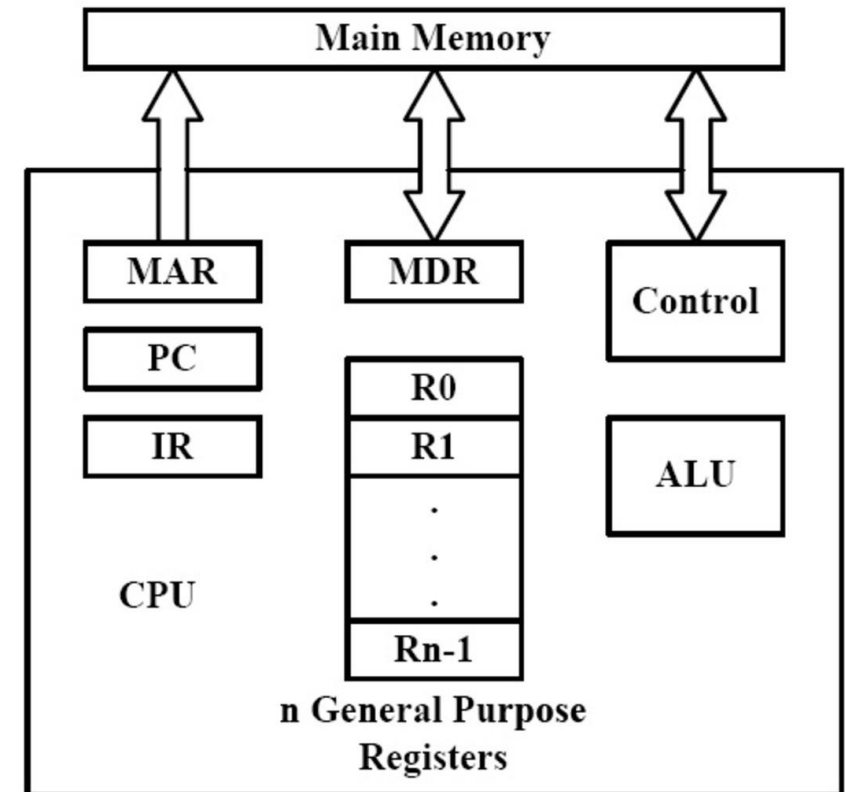
Basic Operational Concepts

- In addition to ALU and CU circuitry, processor has special purpose and general purpose registers
- **Instruction register (IR)** - Holds the instruction that is currently being executed.
- **Program counter (PC)** – Holds the memory address of the next instruction to be fetched and executed. During the execution of an instruction, the content of the PC are updated to the address of the next instruction to be executed.



Basic Operational Concepts

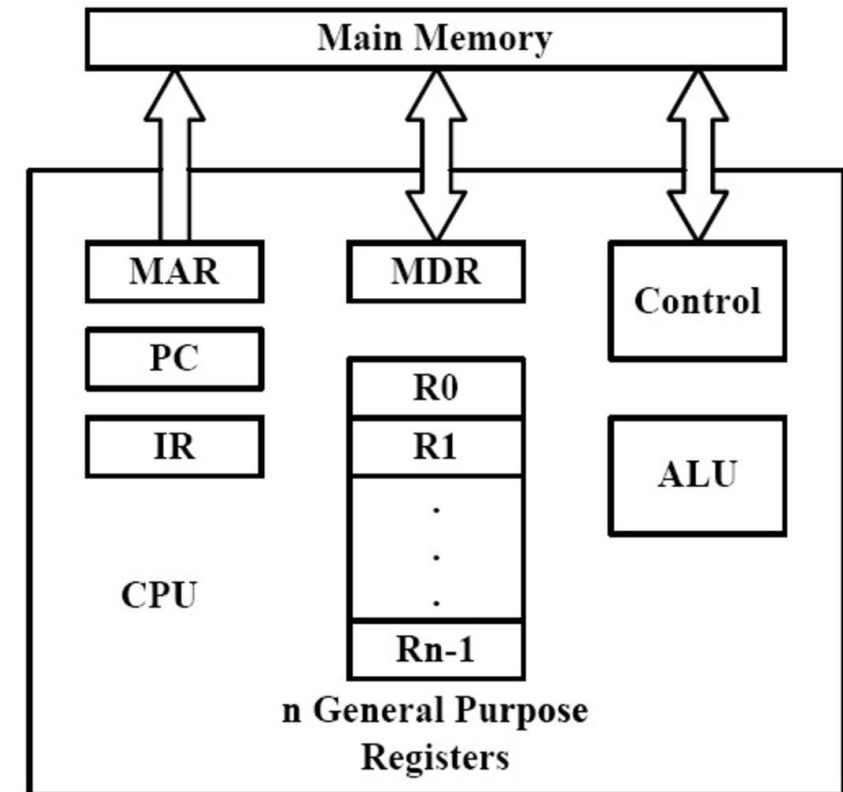
- To facilitate the communication with memory and processor
 - Memory address register (MAR) – Holds the address of the location to be accessed.
 - Memory data register (MDR) – Contains the data to be written into or read out of the addressed location



Basic Operational Concepts

Steps to Fetch Instruction from memory

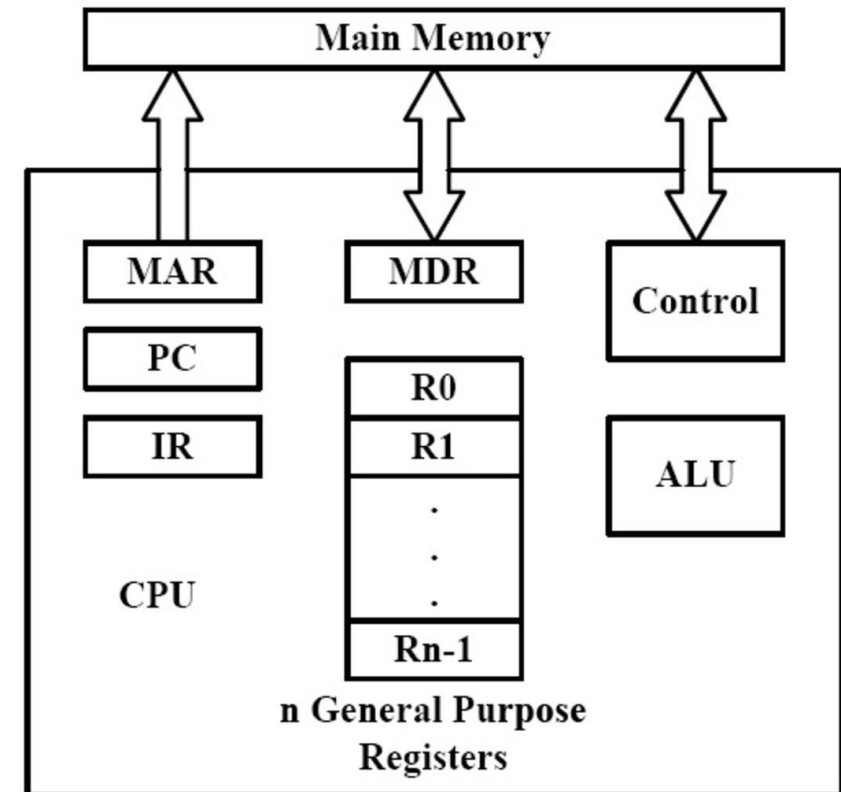
1. Program - stored in memory through input unit
2. PC is set to point to the first instruction (holds the address of the first instruction)
3. The contents of PC are transferred to MAR and Read control signal is sent to the memory
4. The instruction in the corresponding address (MAR) is fetched from memory and loaded into MDR.
5. The contents of MDR are transferred to IR
6. Instruction - ready for execution



Basic Operational Concepts

Steps to Fetch Data

- If operand is in memory, the corresponding address should be sent to MAR – initiating Read Cycle.
- Data fetched from memory and stored in MDR, then to ALU
- Perform operation in ALU
- Store the result back in the memory
 - To general-purpose register
 - To memory (address to MAR, result to MDR – Write in memory in specified location)
- PC is incremented to fetch the next instruction



Basic Operational Concepts

- **Interrupt** – It is a request from an I/O device for the service by the processor
 1. Normal execution of programs may be **preempted** if some device requires **urgent servicing**.
 2. The normal execution of the current program must be interrupted – the device raises an **interrupt signal**.
 3. Processor provide requested service by executing an **Interrupt-service routine (ISR)**.
 4. Current system information backup and restore (PC, general-purpose registers, control information, specific information)
 5. When ISR is completed, program execution is continued.

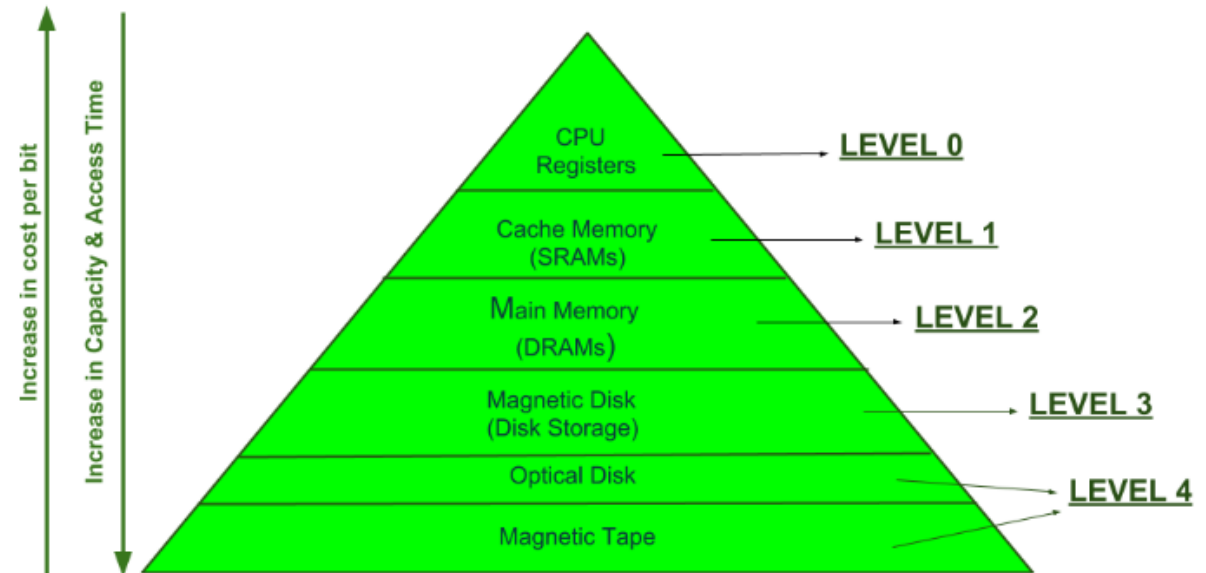
Registers and Register Files

Registers:

- To speed up the processor operations, the processor includes some internal memory storage locations called registers.

- Memory hierarchy

1. Registers
2. Cache memory
3. Main memory
4. Secondary memory



- At higher level of hierarchy, memory is faster, more expensive.
- Registers are top level of hierarchy.

Registers and Register Files

Registers:

- Registers are **temporary storage** locations **inside the processor** circuit that **hold data and addresses**.
- **Some registers are used internally** (by processor only) and cannot be accessed outside the processor, while others are **user-accessible**.
- Most modern processor architectures include both types of registers
- **Internal registers** include the program counter (PC), instruction register (IR), memory data register (MDR), and memory address register (MAR)
- The **storage time** of internal registers is extremely **temporary**, as they often hold data for less than a millisecond
- The **register file** is the component that contains all the general purpose registers of the processor.

Registers – User Visible Registers

- **Register - Two roles**

- **User visible registers** – referenced by machine instruction – visible to the users
- **Control and Status registers** – employed to control the operation – Not visible to the users

- **User-Visible Registers**

- General Purpose Registers
- Data Registers
- Address Registers
- Condition Codes Registers

Registers – User Visible Registers

- **General Purpose Registers**

- True GP Registers

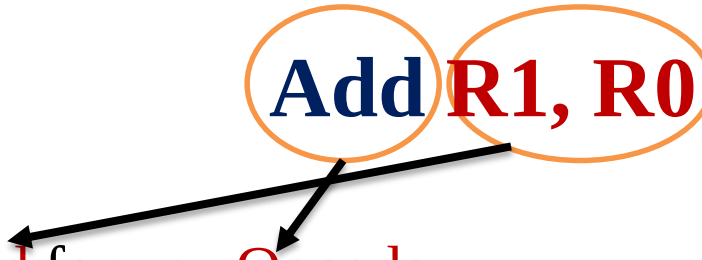
- Registers that can contain the **operand** for any **Opcode**.

- Restricted / Dedicated GP Registers

- Used for specific operations like floating point and stack operations (dedicated registers).

- **Data Registers** - Used only to hold data and cannot be employed in the calculation of an operand address.

- **Accumulators (AC)** - temporary storage location which holds intermediate value in arithmetic and logic operation results.



The diagram shows the instruction 'Add R1, R0'. The word 'Add' is circled in blue, and 'R1, R0' is circled in red. Two black arrows originate from the bottom of the 'Add' circle: one points to the word 'operand' in the text below, and the other points to the word 'Opcode' in the text below.

Registers – User Visible Registers

- **Address Registers** – devoted for particular addressing mode.
 - **Segment registers** - holds the address of the base segment.
 - **Index registers** – used for indexed addressing and may be auto-indexed (increment).
 - **Stack pointer (SP)** – points to the top of the stack.
- **Condition Codes (Flag) Registers**
 - FLAG register is the **status register** that contains the current state of the processor (reflects the result of arithmetic operation).
 - Machine instructions allow these bits to be read by implicit reference (No explicit call needed).
 - Programmers cannot alter them.
 - Example: In some machines, sub-routine calls will result in automatic saving of all user visible registers, to be restored on return.

Registers – User Visible Registers

- **Condition Codes (Flag) Registers**

- CCR contains **five flag bits** – set by CPU Hardware (ALU)– to hold information about the result of an arithmetic or logical operation instruction that has just been executed.

- **Flag Bits:**

- **Carry C:** Set to 1 if an add operation produces a carry or a subtract operation produces a borrow; otherwise cleared to 0.
- **Overflow V:** Useful only during operations on signed integers.
- **Zero Z:** Set to 1 if the result is 0, otherwise cleared to 0.
- **Negate N:** Meaningful only in signed number operations. Set to 1 if a negative result is produced.
- **Extend X:** Functions as a carry for multiple precision arithmetic operations.

Registers – Control and Status registers

- **Control and Status Registers**

- Not visible to the user, May be visible in a control or operating system mode (supervisory mode).
- Program counter (PC) holds the address of the instruction that should be executed next.
- Instruction register(IR) holds the actual instruction to be executed.
- Memory Address Register (MAR) connects to address bus
- Memory Buffer Register (MBR) connects to data bus, feeds other registers.

Example of Register Organizations

Data Registers	
D0	
D1	
D2	
D3	
D4	
D5	
D6	
D7	

Address Registers	
A0	
A1	
A2	
A3	
A4	
A5	
A6	
A7	
A7'	

Program Status	
Program Counter	
Status Register	

(a) MC68000

General Registers

AX	Accumulator
BX	Base
CX	Count
DX	Data

Pointer & Index

SP	Stack Pointer
BP	Base Pointer
SI	Source Index
DI	Dest Index

Segment

CS	Code
DS	Data
SS	Stack
ES	Extra

Program Status

Instr Ptr
Flags

(b) 8086

General Registers

EAX	AX
EBX	BX
ECX	CX
EDX	DX

ESP	SP
EBP	BP
ESI	SI
EDI	DI

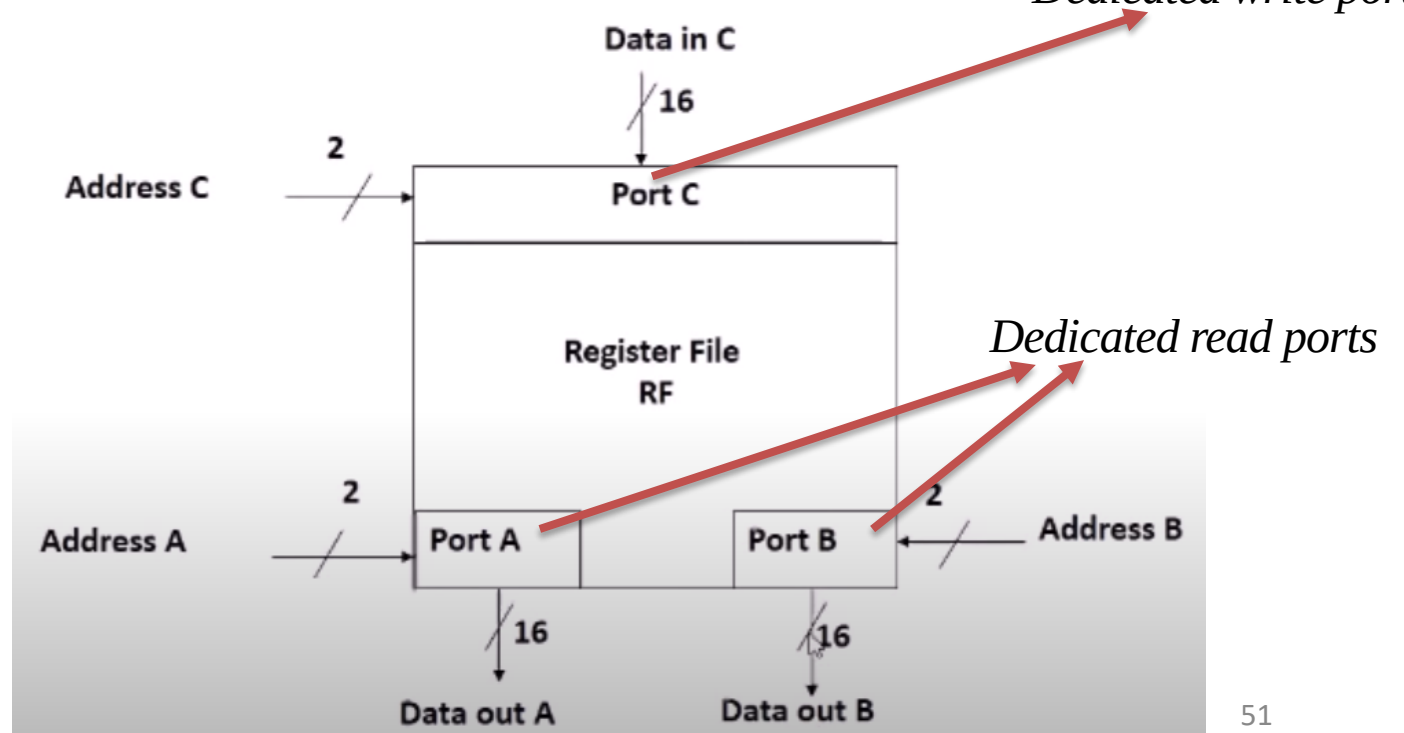
Program Status

FLAGS Register
Instruction Pointer

(c) 80386 - Pentium II

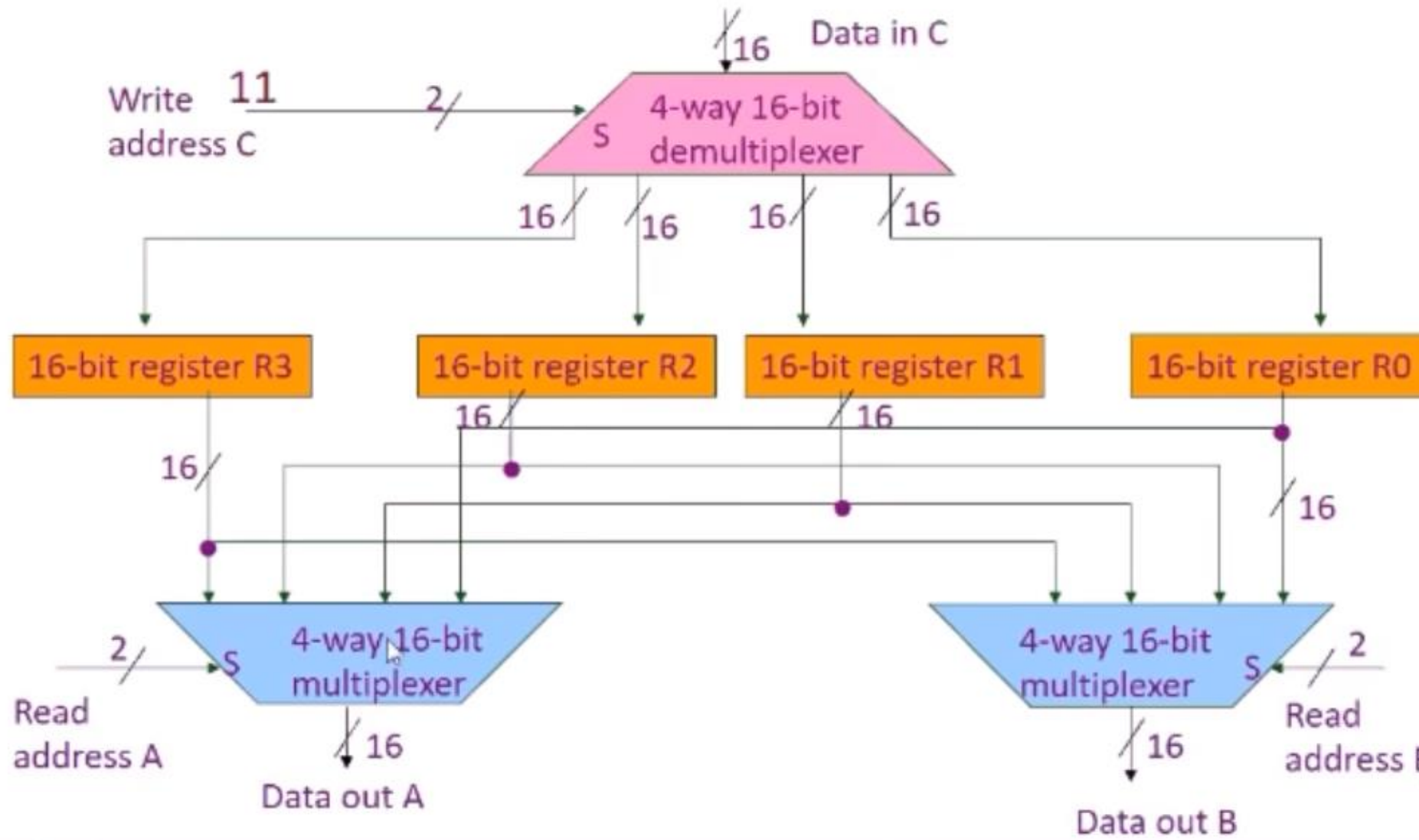
Register Files

- A register file is an **array of processor registers (Array of GP registres)** in a CPU.
- Implemented by way of **fast static RAMs with multiple ports**. *Dedicated read and write ports*
- Ordinary multi-ported SRAMs will usually read and write through the same ports.
- **Example: Register file with 3 access ports**



Register Files

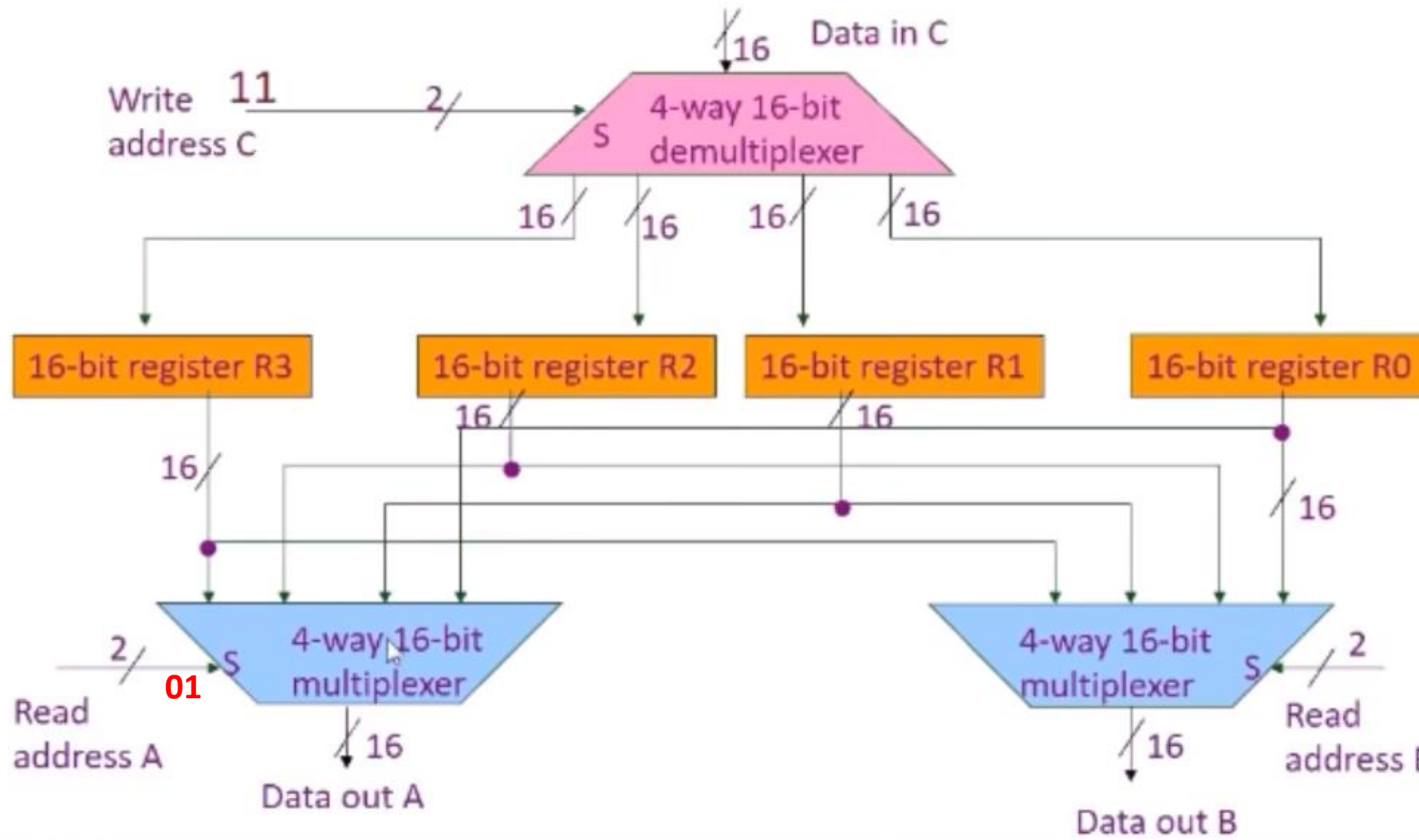
- Example: Register file with 3 access ports – Logic Diagram



Ex: $R3 \leftarrow R1 + R2$
Read Address A = 01
Read Address B = 10
Write Address C = 11

Register Files

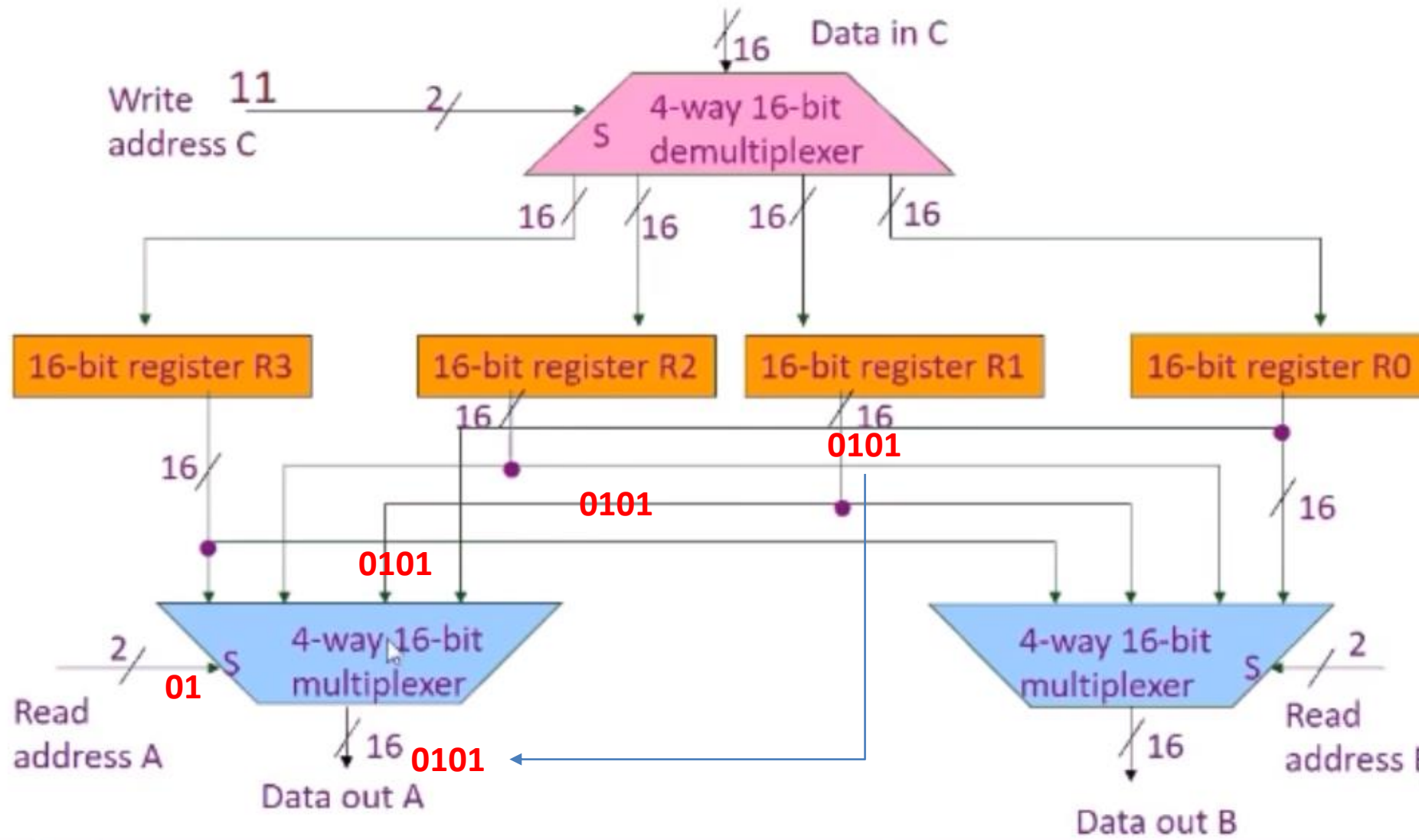
- Example: Register file with 3 access ports – Logic Diagram



Ex: $R3 \leftarrow R1 + R2$
Read Address A = 01
Read Address B = 10
Write Address C = 11

Register Files

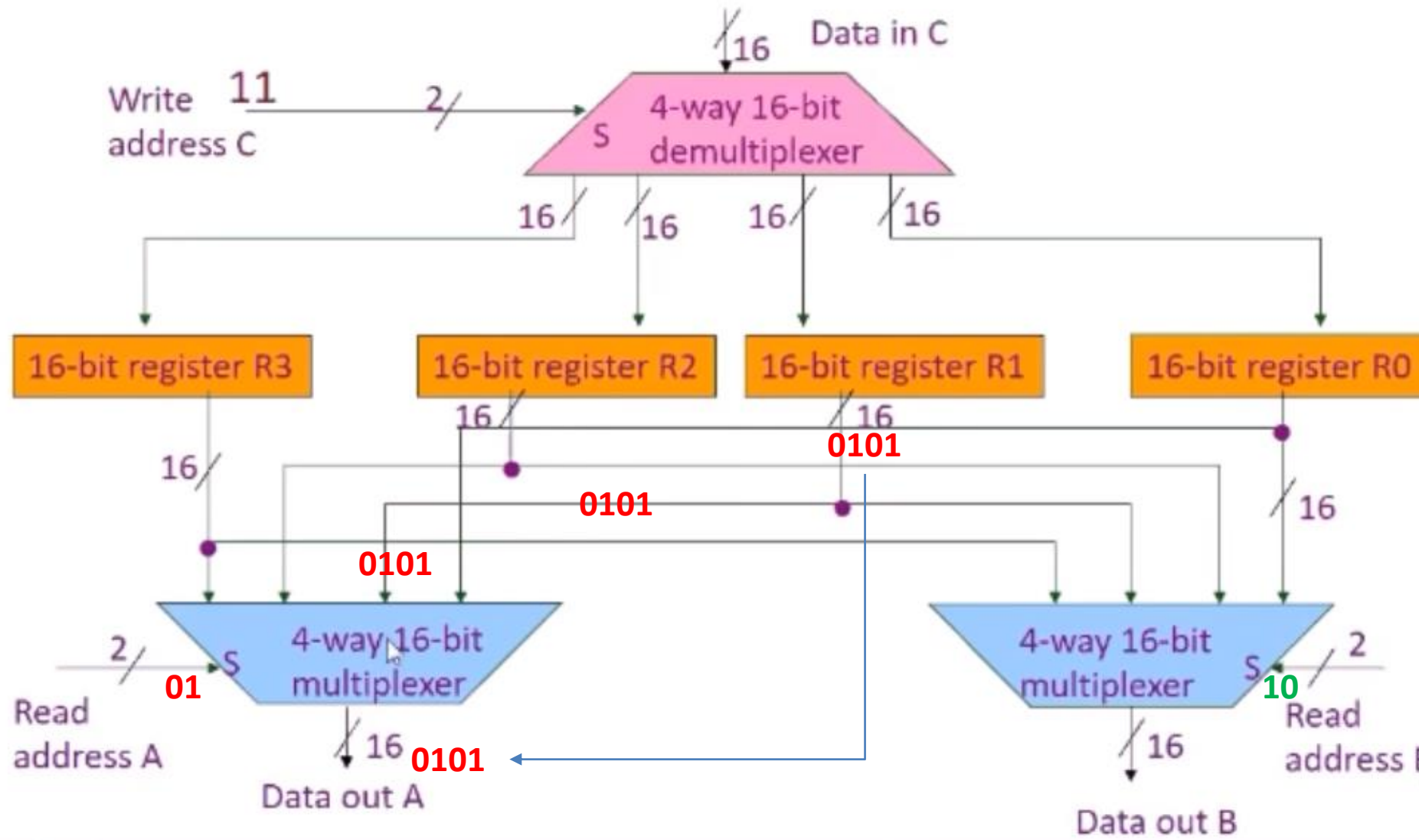
- Example: Register file with 3 access ports – Logic Diagram



Ex: $R3 \leftarrow R1 + R2$
Read Address A = 01
Read Address B = 10
Write Address C = 11

Register Files

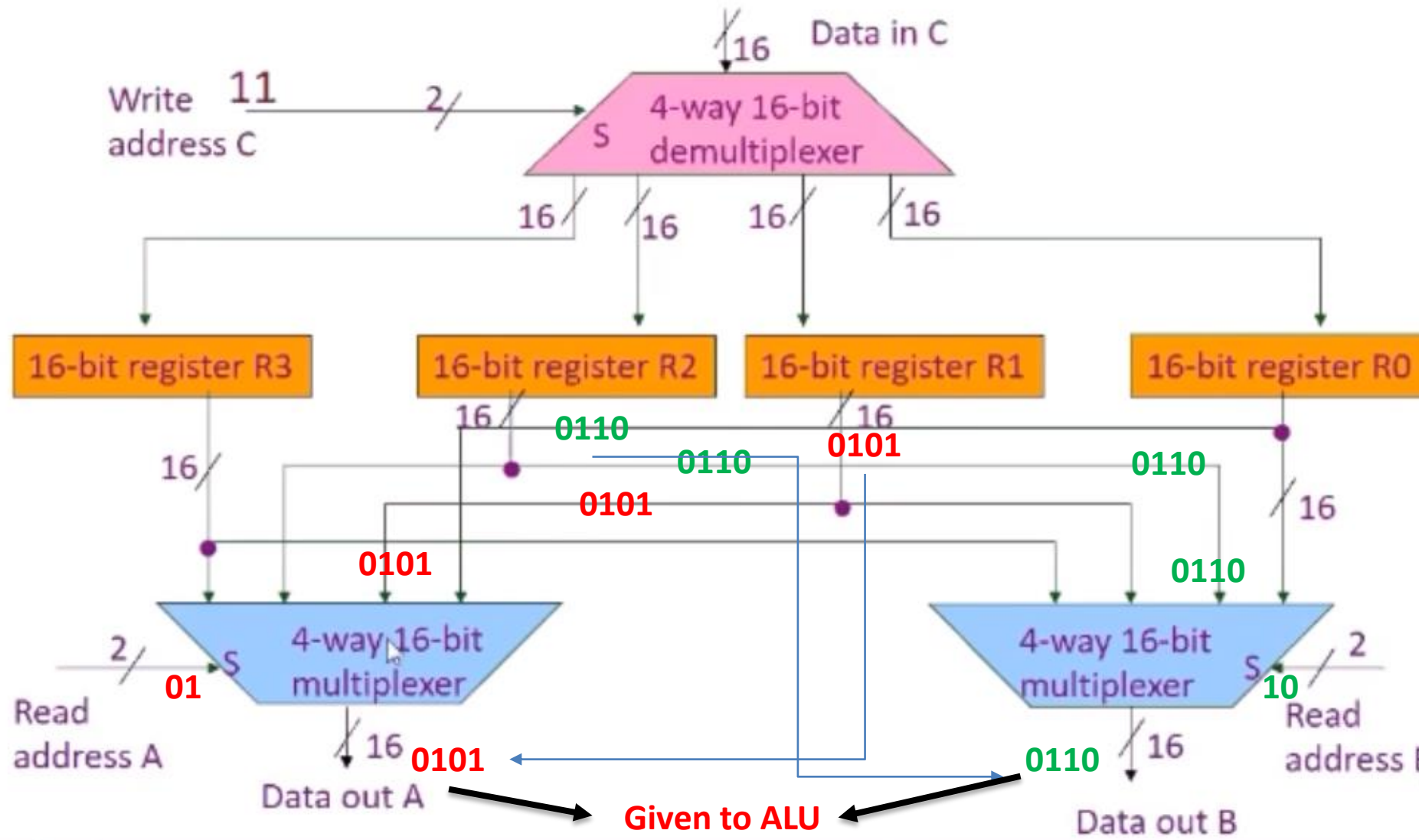
- Example: Register file with 3 access ports – Logic Diagram



Ex: $R3 \leftarrow R1 + R2$
Read Address A = 01
Read Address B = 10
Write Address C = 11

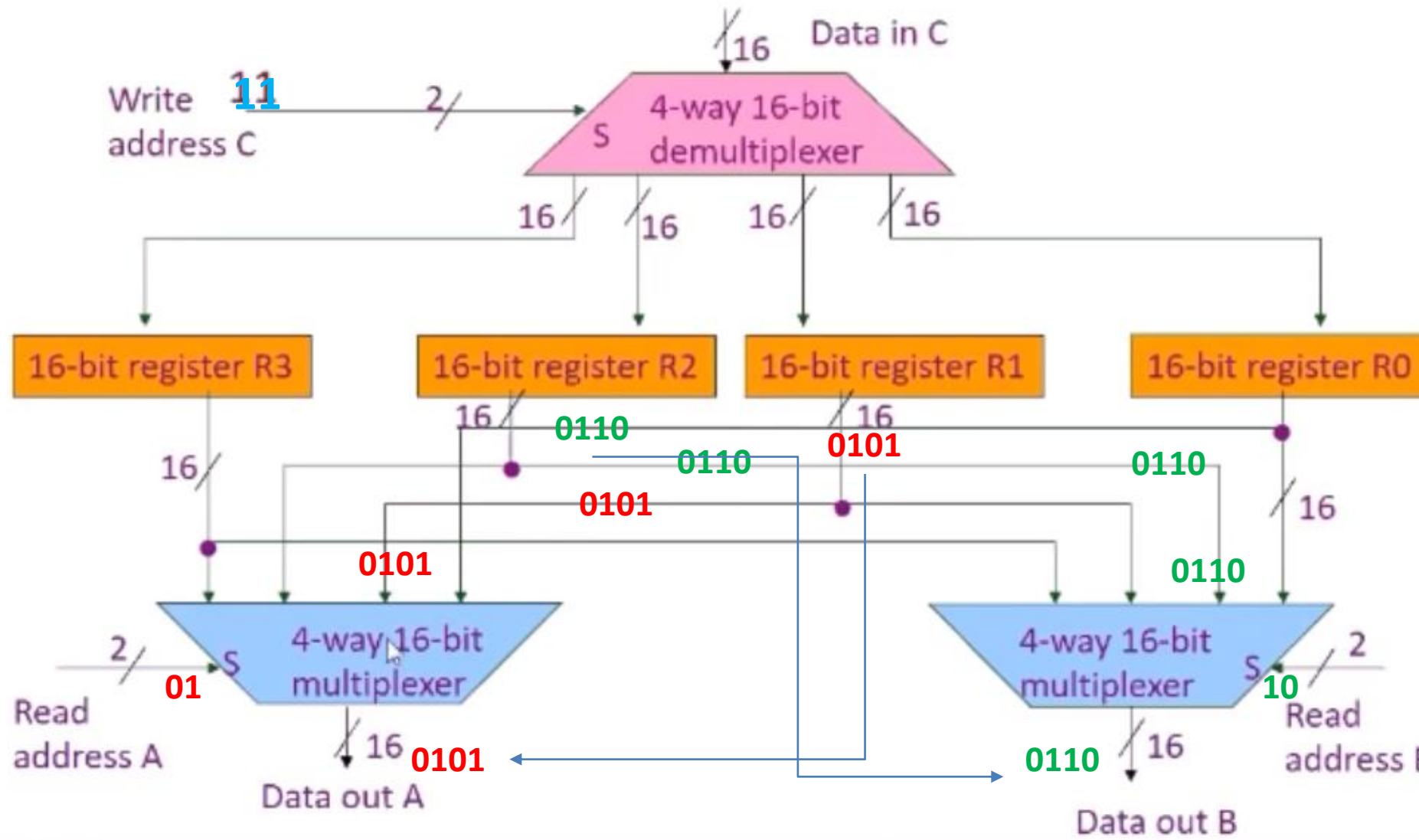
Register Files

- Example: Register file with 3 access ports – Logic Diagram



Register Files

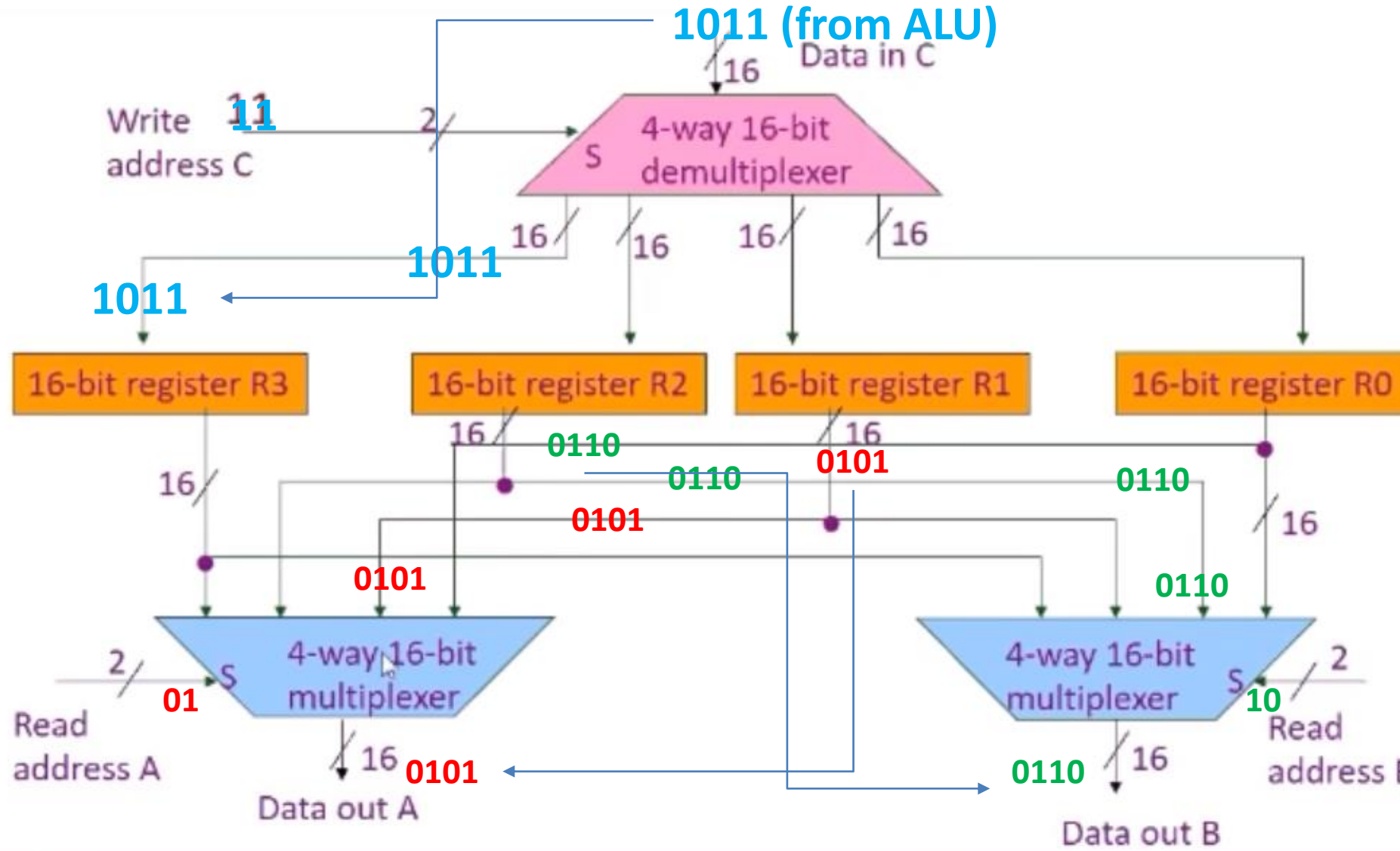
- Example: Register file with 3 access ports – Logic Diagram



Ex: $R3 \leftarrow R1 + R2$
Read Address A = 01
Read Address B = 10
Write Address C = 11

Register Files

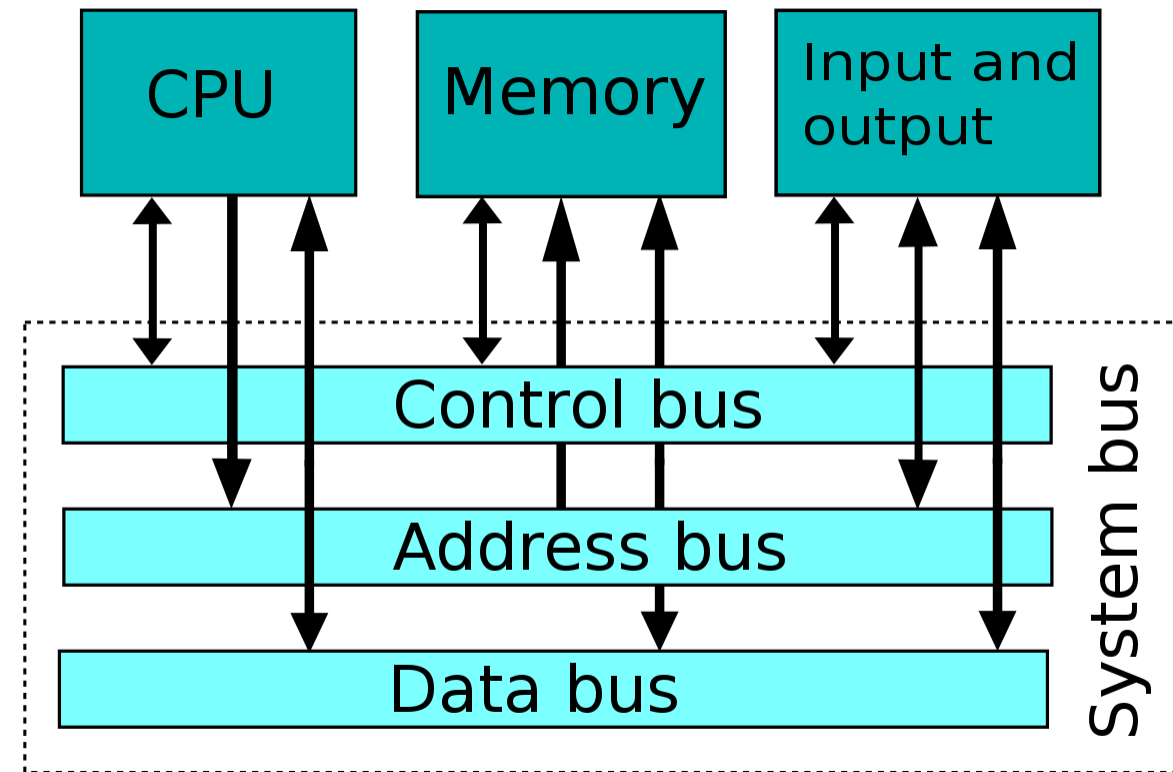
- Example: Register file with 3 access ports – Logic Diagram



Ex: $R3 \leftarrow R1 + R2$
Read Address A = 01
Read Address B = 10
Write Address C = 11

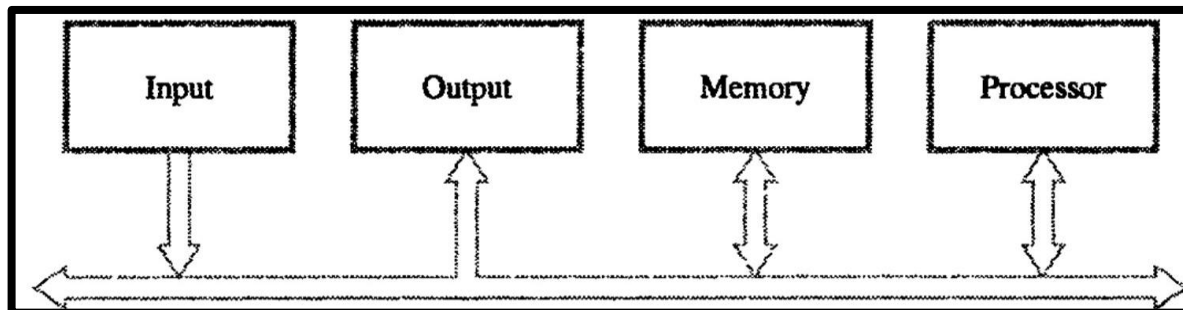
Interconnection of Components - Bus Structures

- All five functional units must be connected in some way to form an operational system.
- There are many ways to connect different parts inside a computer together.
- **Bus**
 - A group of lines that serves as a connecting path for several devices.
 - Group of conducting lines that carries data, address and control signals.
 - Data Bus, Address Bus, Control Bus

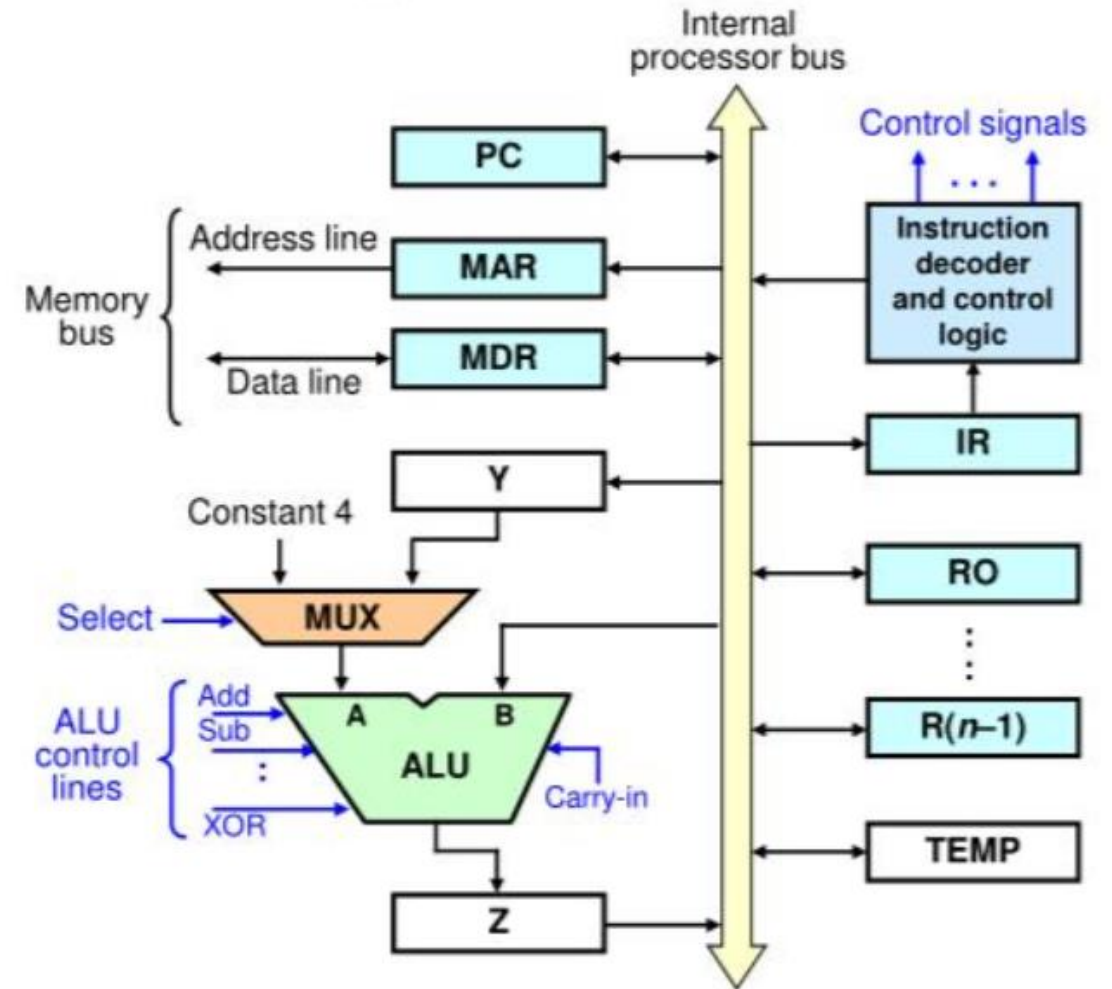


Interconnection of Components - Bus Structures

- The simplest way to interconnect functional units is to use **single bus**.
- It can support only **one transfer at a time**.
- Only two units can actively use the bus at any given time.
- **Advantages:** Simple, low cost, flexibility for attaching peripheral devices.

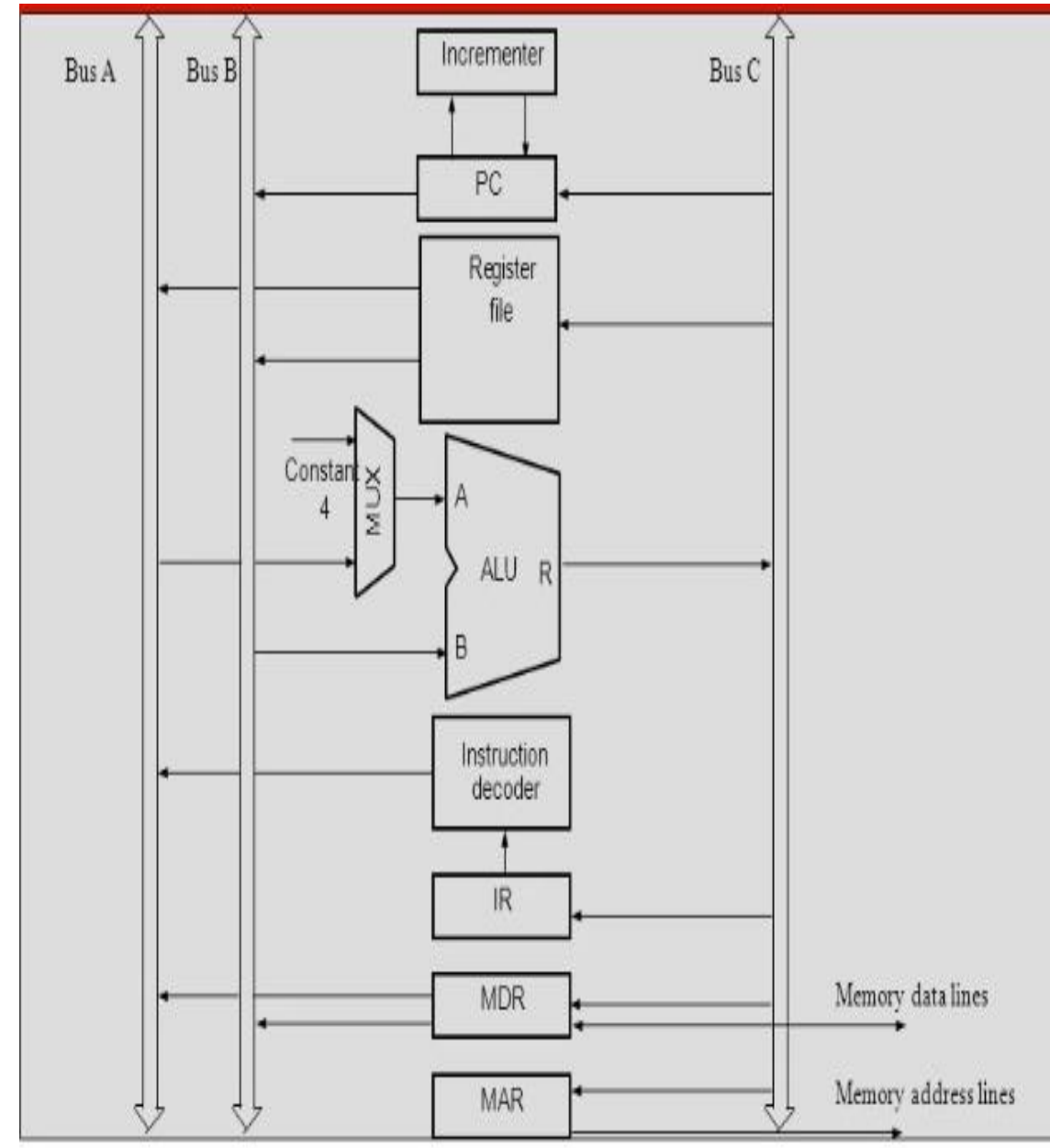
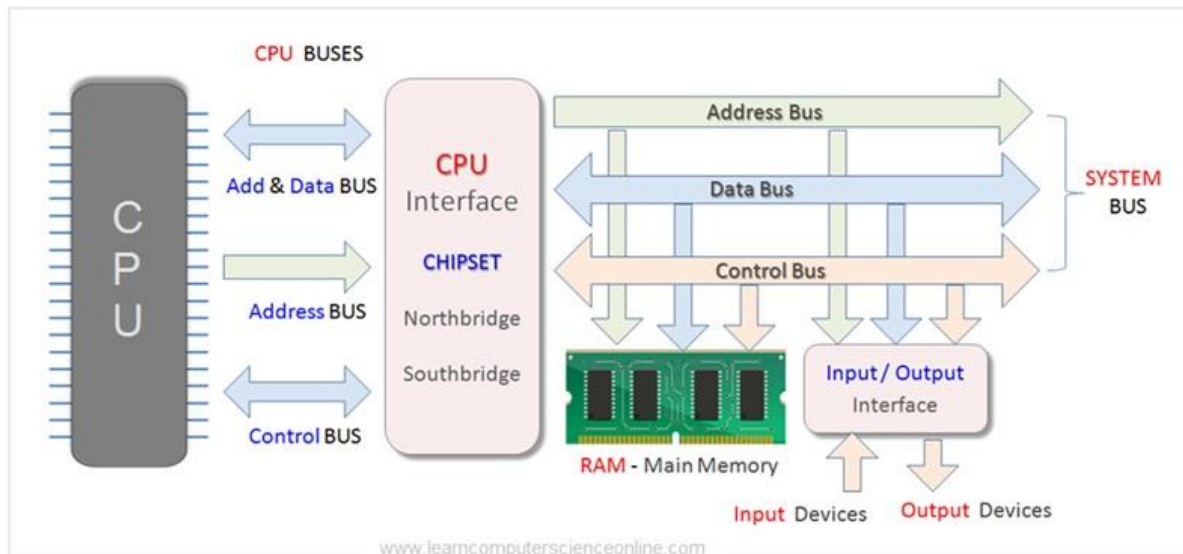


Single-bus Organization



Interconnection of Components - Bus Structures

- **Multiple Bus structure** achieve more concurrency in operations by allowing two or more transfers to be carried out at the same time.
- It provides **better performance** at an increased cost and complexity.



Interconnection of Components - Bus Structures

- **Speed Issues**
- Different devices have **different transfer/operate speed**.
- If the speed of bus is bounded by the **slowest device** connected to it, the efficiency will be very low.
- How to solve this?
- A common approach – **Use buffer registers**.
- Buffer registers are used to prevent a high speed processor from being locked to a slow I/O device during a sequence of data transfer.

Instruction Formats

- **In designing an instruction set, consideration is given to**
 - **Instruction length** – whether short, long or variable
 - **Number of Operands**
 - **Number of addressable registers**
 - **Memory Organization** – byte addressable or Word addressable
 - **Addressing Modes** – direct, indirect or indexed

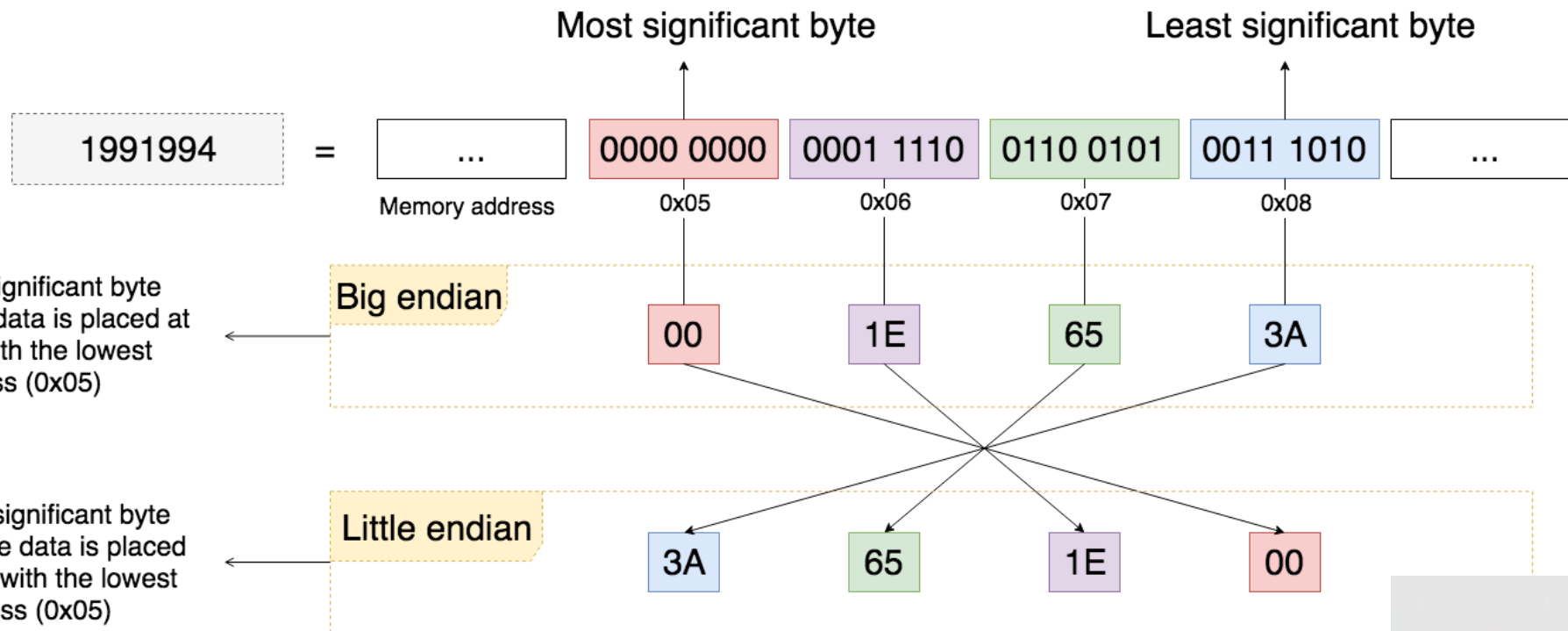
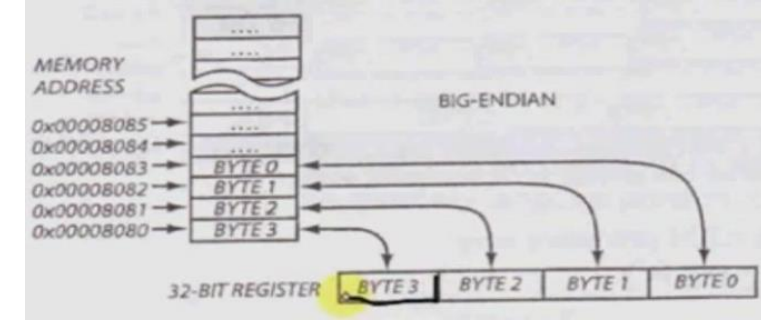
Byte Ordering (Big Endian and Little Endian Formats)

Terminologies

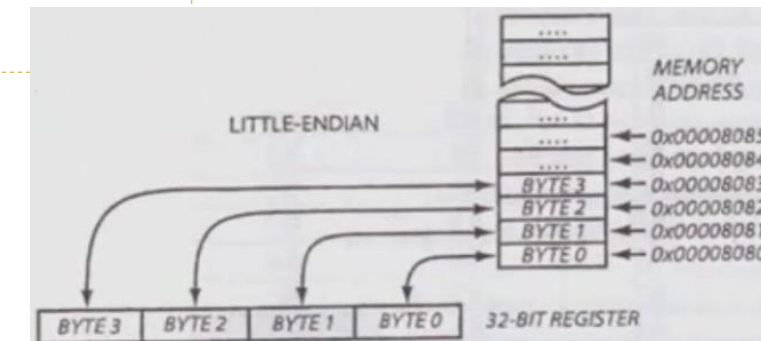
- **Bit** - the smallest piece of information
 - represents a digit of the binary numeral system
- **Byte** - A string of 8 bits
- Two ways to store a string of data (Bytes) in computers:
 - Big Endian
 - Little Endian
- **Least Significant Byte (LSB)** is the right-most bit in a string - because it has the least effect on the value of the binary number.
- **Most Significant Byte (MSB)** - the left-most byte that carries the greatest numerical value.

Big Endian and Little Endian Formats

- In **Big Endian**, the MSB of the data is placed at the byte with the lowest address. (First byte stored in the memory first)



- In **Little Endian**, the LSB of the data is placed at the byte with the lowest address. (Last byte stored in the memory first)



Byte Ordering (Big Endian and Little Endian Formats)

- **Big Endian** - the most common format in data networking (TCP, UDP, IPv4 and IPv6 are using Big endian order to transmit data)
- **Little Endian** - on microprocessors

- **Show the big endian and small endian arrangements of the bytes in 12345678_{16}**

Address →	00	01	10	11
Big Endian	12	34	56	78
Little Endian	78	56	34	12

CPU Organization

- Classified on the basis of the **number of address fields**

- Single Accumulator
 - Result usually goes to the Accumulator
 - Accumulator has to be saved to memory quite often
- General Register
 - Registers hold operands thus reduce memory traffic
 - Register bookkeeping
- Stack
 - Operands and result are always in the stack

*involves a special register called the **accumulator**.*

***multiple registers** used for the computation purpose*

*work on **stack basis operation** due to which it does not contain any address field*

Comparison of Architectures

- **Stack architecture**, instructions and operands are implicitly taken from the stack.
 - A stack cannot be accessed randomly.
- **Accumulator architecture**
 - one operand of a binary operation is implicitly in the accumulator.
 - One operand is in memory, creating lots of bus traffic.
- **General purpose register (GPR) architecture**, registers can be used instead of memory.
 - Faster than accumulator architecture.
 - Efficient implementation for compilers.
 - Results in longer instructions.

Instruction Formats

- Three-Address Instructions

- ADD R1, R2, R3

$$R1 \leftarrow [R2] + [R3]$$

- Two-Address Instructions

- ADD R1, R2

$$R1 \leftarrow [R1] + [R2]$$

- One-Address Instructions

- ADD M

$$AC \leftarrow [AC] + M[AR]$$

- Zero-Address Instructions

- ADD

$$TOS \leftarrow [TOS] + [TOS - 1]$$

- RISC Instructions

- Lots of registers. Memory is restricted to Load & Store

First operand - Destination



Stack Architecture

- Stack architectures require us to think about arithmetic expressions a little differently.
- We are **accustomed** to writing expressions using *infix* notation
 - such as: $Z = X + Y$
- Stack arithmetic requires that we use *postfix* notation

$$Z = XY+$$

- The principal advantage of postfix notation is that parentheses are not used.
- For example, the infix expression,

$$Z = (X \times Y) + (W \times U),$$

becomes:

$$Z = X Y \times W U \times + \text{ in postfix notation}$$

Stack Architecture

- In a stack ISA, the postfix expression,

$$Z = X Y \times W U \times +$$

might look like this:

```
PUSH X  
PUSH Y  
MULT  
PUSH W  
PUSH U  
MULT  
ADD  
PUSH Z
```

Note: The result of a binary operation is implicitly stored on the top of the stack!

Instruction Formats

Example: Evaluate $(A+B) * (C+D)$

- Three-Address**

CPU Organization :
General Register

1. ADD R1, A, B; $R1 \leftarrow M[A] + M[B]$
2. ADD R2, C, D; $\longrightarrow$ $R2 \leftarrow M[C] + M[D];$
3. MUL X, R1, R2; $M[X] \leftarrow R1 * R2$

LOAD, STORE, MOV instructions - Not allowed

Instruction Formats

Example: Evaluate $(A+B) * (C+D)$

- Two-Address**

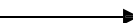
CPU Organization :
General Register

1. MOV	R1, A ;	$R1 \leftarrow M[A]$
2. ADD	R1, B ;	$R1 \leftarrow R1 + M[B]$
3. MOV	R2, C ;	$R2 \leftarrow M[C]$
4. ADD	R2, D ;	$R2 \leftarrow R2 + M[D]$
5. MUL	R1, R2 ;	$R1 \leftarrow R1 * R2$
6. MOV	X, R1 ;	$M[X] \leftarrow R1$

MOV – Loads the
Data from Memory to
GP Registers (Except
Accumulator & MQ)
&
Viceversa

Only MOV instruction allowed

LOAD, STORE operations – Not allowed



Instruction Formats

Example: Evaluate $(A+B) * (C+D)$

- One - Address**

1. LOAD A

$AC \leftarrow M[A]$

2. ADD B

$AC \leftarrow AC + M[B]$

3. STORE T

$M[T] \leftarrow AC$

4. LOAD C

$AC \leftarrow M[C]$

5. ADD D

$AC \leftarrow AC + M[D]$

6. MUL T

$AC \leftarrow AC * M[T]$

7. STORE X

$M[X] \leftarrow AC$

CPU Organization :
Accumulator

LOAD – Loads the Data from Memory to **Accumulator**

STORE – Stores the data from **Accumulator** to Memory Location

MOV instruction – Not allowed

Only LOAD, STORE operations allowed

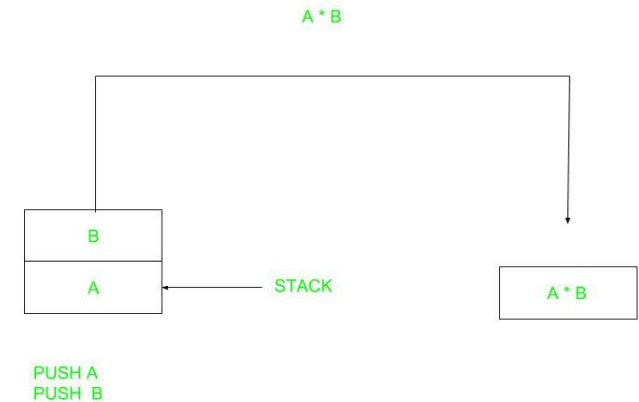
Instruction Formats

Example: Evaluate $(A+B) * (C+D)$

- Zero - Address**

- | | |
|------------|------------------------------|
| 1. PUSH A; | $TOS \leftarrow A$ |
| 2. PUSH B; | $TOS \leftarrow B$ |
| 3. ADD; | $TOS \leftarrow (A + B)$ |
| 4. PUSH C; | $TOS \leftarrow C$ |
| 5. PUSH D; | $TOS \leftarrow D$ |
| 6. ADD; | $TOS \leftarrow (C + D)$ |
| 7. MUL; | $TOS \leftarrow (C+D)*(A+B)$ |
| 8. POP X; | $M[X] \leftarrow TOS$ |

CPU Organization :
Stack



Disadvantages of Addresses

*choice of instruction format - depends on the specific requirements of the **computer architecture** and the trade-offs between **code size, execution time, and flexibility**.*

- **Zero-address instructions**

- They can be limited in their functionality and **do not allow for much flexibility** in terms of addressing modes or operand types.

- **One-address instructions**

- They can be **slower to execute** since they require operand fetching and addressing.

- **Two-address instructions**

- They require **more memory space** than one-address instructions and can be **slower** to execute since they require operand fetching and addressing.

- **Three-address instructions**

- They require **even more memory space** than two-address instructions and can be **slower to execute** since they require operand fetching and addressing.

Example Instructions

$$Y = \frac{A - B}{C + (D \times E)}$$

Instruction		Comment
SUB	Y, A, B	$Y \leftarrow A - B$
MPY	T, D, E	$T \leftarrow D \times E$
ADD	T, T, C	$T \leftarrow T + C$
DIV	Y, Y, T	$Y \leftarrow Y \div T$

(a) Three-address instructions

Instruction		Comment
MOVE	Y, A	$Y \leftarrow A$
SUB	Y, B	$Y \leftarrow Y - B$
MOVE	T, D	$T \leftarrow D$
MPY	T, E	$T \leftarrow T \times E$
ADD	T, C	$T \leftarrow T + C$
DIV	Y, T	$Y \leftarrow Y \div T$

(b) Two-address instructions

Instruction		Comment
LOAD	D	$AC \leftarrow D$
MPY	E	$AC \leftarrow AC \times E$
ADD	C	$AC \leftarrow AC + C$
STOR	Y	$Y \leftarrow AC$
LOAD	A	$AC \leftarrow A$
SUB	B	$AC \leftarrow AC - B$
DIV	Y	$AC \leftarrow AC \div Y$
STOR	Y	$Y \leftarrow AC$

(c) One-address instructions

Example Instructions

In a two-address ISA, (e.g., Intel, Motorola), the infix expression,

$$Z = X \times Y + W \times U$$

might look like this:

```
LOAD R1,X
MULT R1,Y
LOAD R2,W
MULT R2,U
ADD R1,R2
STORE Z,R1
```

Note: One-address ISAs usually require one operand to be a register.

With a three-address ISA, (e.g., mainframes), the infix expression,

$$Z = X \times Y + W \times U$$

might look like this:

```
MULT R1,X,Y
MULT R2,W,U
ADD Z,R1,R2
```

In a one-address ISA, like MARIE, the infix expression,

$$Z = X \times Y + W \times U$$

looks like this:

```
LOAD X
MULT Y
STORE TEMP
LOAD W
MULT U
ADD TEMP
STORE Z
```

Example Instructions

- Write a program to evaluate the following arithmetic statement
$$X = [A (B + C) - D] / (E + F - G)$$
 - i) using a general register computer with three-address and two-address instructions
 - ii) using an accumulator type computer with one-address instructions
 - iii) using a stack organized computer with zero-address operation instructions.

- Write a program to evaluate the arithmetic statement

$$X = \frac{(A-B)*(C+D+E)}{(F-G)*(H-I)}$$

- a. Using general register with 3 address instructions
- b. Using general register with 2 address instructions

Organization of Von Neumann Machine (IAS Computer)

- Developed by John Von Neumann in 1940 at Princeton University.
- **IAS** - Institute for Advanced Studies.
- The task of entering and altering programs for **ENIAC** was extremely tedious
- **ENIAC** - (Electronic Numerical Integrator And Computer) - world's first general-purpose electronic computer - the world's first programmable computer
 - **Disadvantage:** programming the computer was a **complex, manual process** that could take days.

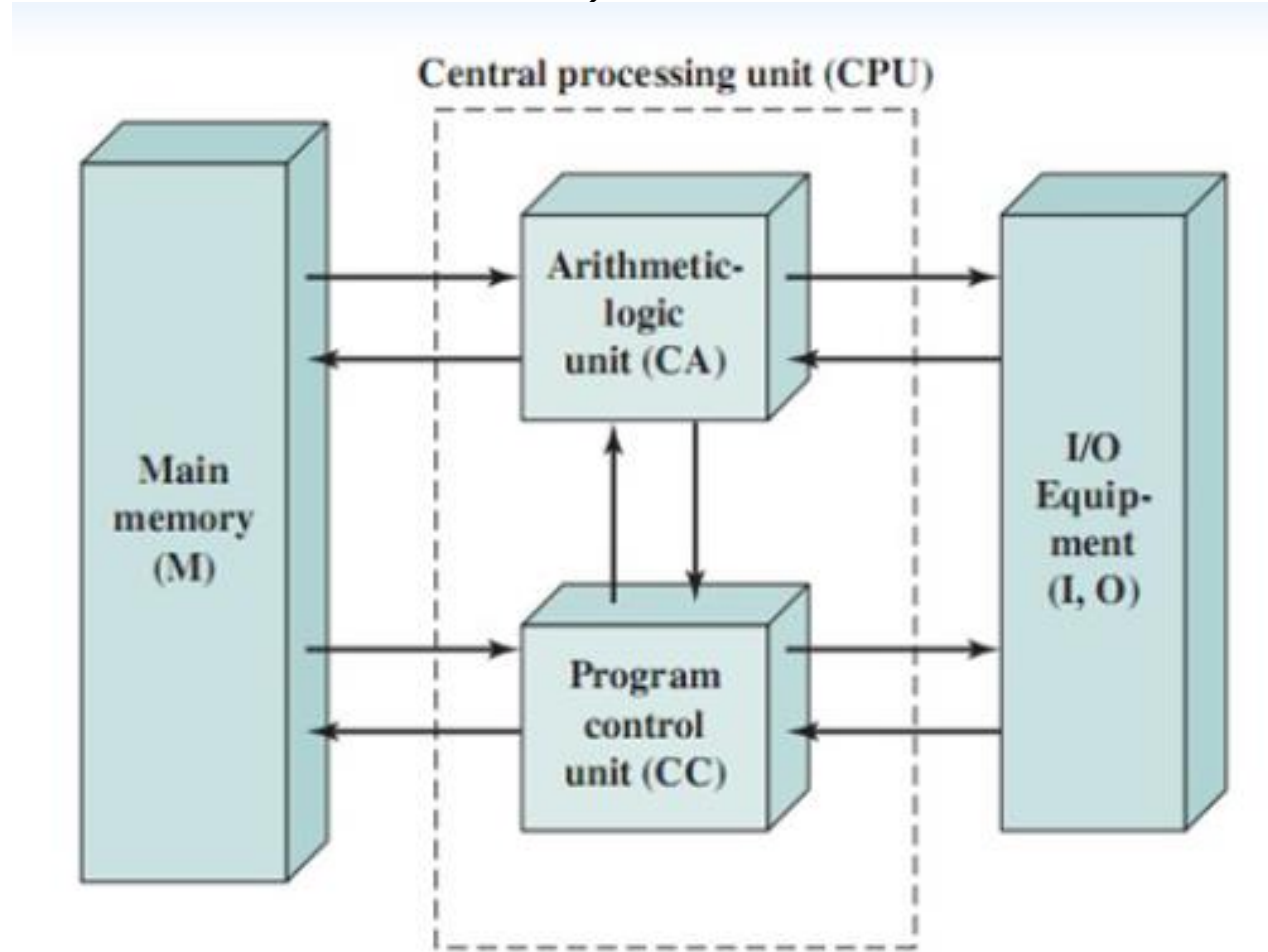
Organization of Von Neumann Machine (IAS Computer)

- **IAS computer** is the **father of all modern computers**
 - **Primary concept** - storing the program in memory.
- **Stored program concept** – program stored in the computer along with any relevant data
- A **stored program computer** consists of a **processing unit** and an attached **memory** system.
 - The **processing unit** consists of **data-path and control**.
 - The **data-path** contains registers to hold data and functional units, such as arithmetic logic units and shifters, to operate on data.

Structure of Von Neumann Machine

*main memory, -
stores both data
and
instructions*

*ALU - capable of operating on binary data (CA: central
arithmetical)*

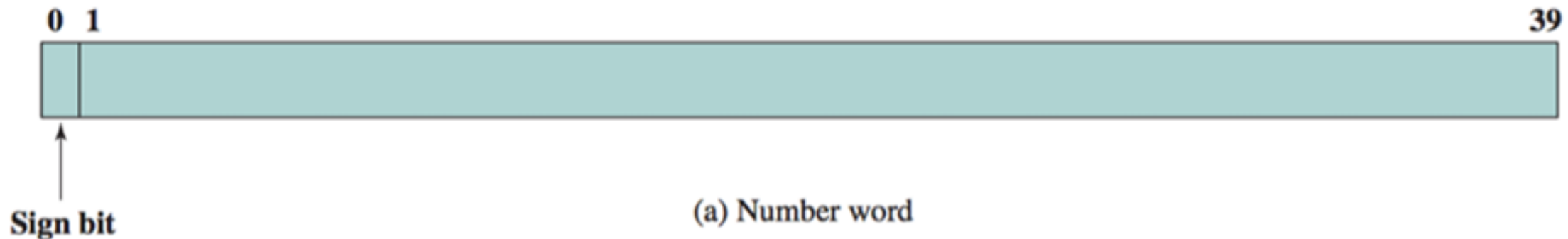


*I/O equipment operated
by the control unit*

*Control unit - interprets the instructions in the memory and
causes them to be executed (CC: central control)*

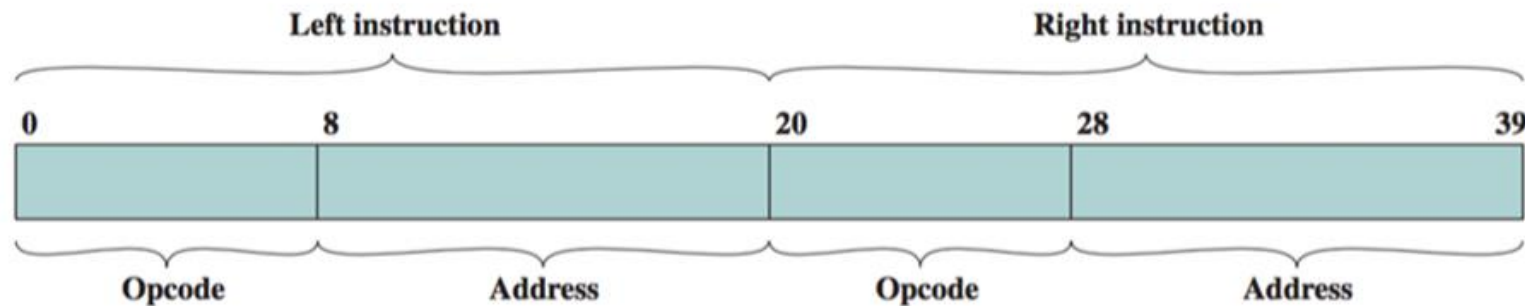
IAS Architecture

- Both data and instructions are stored in the same memory.
- Numbers are represented in binary form, and each instruction is a binary code.
- **Memory Unit** of the IAS consists of
 - 1000 storage locations, called words
 - Each word - 40 binary digits (bits) – 5 bytes
 - **Data Storage:** Each number is represented by 1 sign bit + 39-bit value -- (Total 40 bits)



IAS Architecture

- **Memory Unit** of the IAS consists of
 - **Instruction Storage:** A word may alternatively contain **two 20-bit instructions**
 - Each instruction
 - **An Opcode (8-bits)** - specifying the operation to be performed
 - **An Address (12-bits)** - designating one of the words in memory (numbered from 0 to 999).
 - If Address - Memory address – requires 12 bits
 - If Address - Registers – 4 bits required – remaining 8 bits left unused

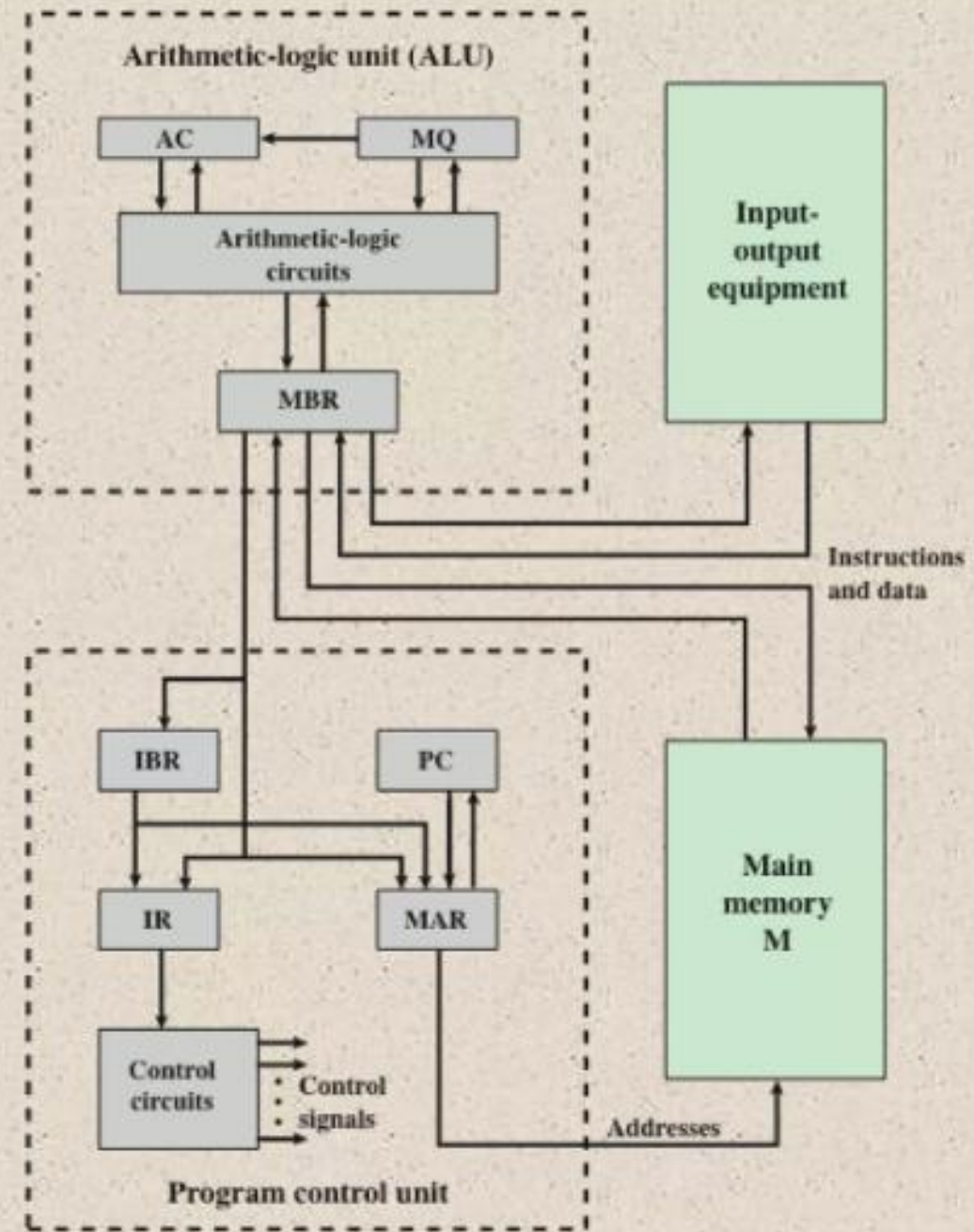


(b) Instruction word

- **Control Unit** operates the IAS by **fetching instructions** from **memory** and executing them one at a time.

Von Neumann Machine (Expanded structure of IAS)

- **Basic Elements**
 - Processor
 - Memory
- **Note:**
 - Each word – 40 bits
 - Two instructions (each 20 bits) stored



IAS Architecture

The **control unit and the ALU** contain storage locations - **registers**, defined as follows:

- **Memory buffer register (MBR)**: Contains a word to be stored in memory or sent to the I/O unit, or is used to receive a word from memory or from the I/O unit.
- **Memory address register (MAR)**: Specifies the address in memory of the word to be written from or read into the MBR.
- **Instruction register (IR)**: Contains the 8-bit opcode instruction being executed.
- **Instruction buffer register (IBR)**: Employed to hold temporarily the righthand instruction from a word in memory.

IAS Architecture

- **Program counter (PC):** Contains the address of the next instruction pair to be fetched from memory.
- **Accumulator (AC) and Multiplier Quotient (MQ):** Employed to hold temporarily operands and results of ALU operations.

For example, the result of **multiplying two 40-bit numbers** is an 80-bit number – (40 bits are stored in the AC and the least significant in the MQ).

Each word – 40 bits. Hence two registers MQ and AC are used for temporarily storing 80 bits. Then transferred to memory and stored in two consecutive 40 bit words in the memory...

Example

MEMORY

- 1. LOAD M(500), ADD M(501)
- 2. STOR M(500), (Other Ins)

.....

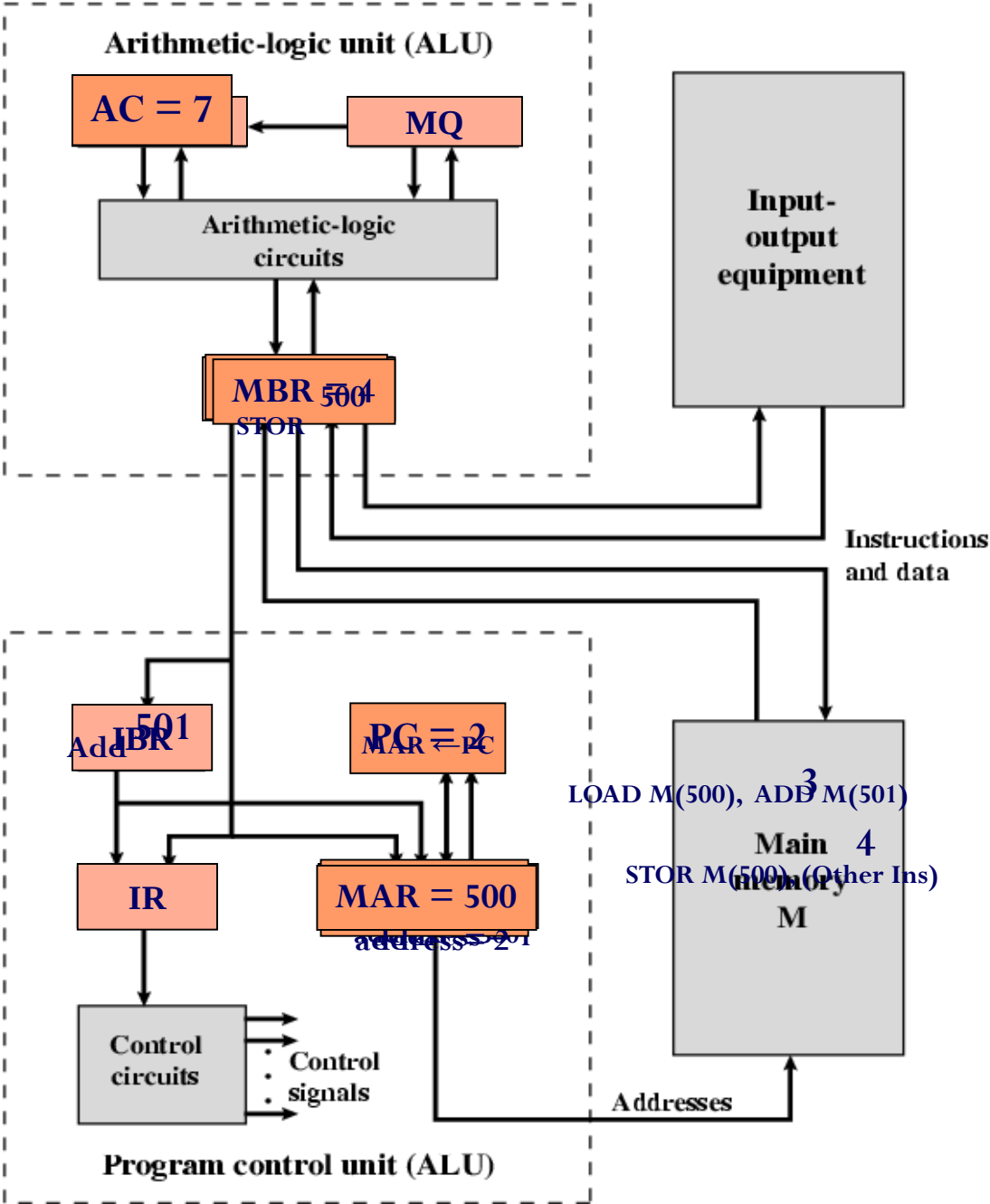
500. 3

501. 4

PC	1
MAR	500
MBR	3
IR	LOAD M(500)
IBR	ADD M(501)
AC	3

PC	2
MAR	501
MBR	4
IR	ADD M(501)
IBR	ADD M(501)
AC	7

PC	2
MAR	500
MBR	7
IR	STOR M(500)
IBR	Other Ins
AC	7



Example

MEMORY

1. LOAD M(500), ADD M(501)
2. STOR M(500), (Other Ins)

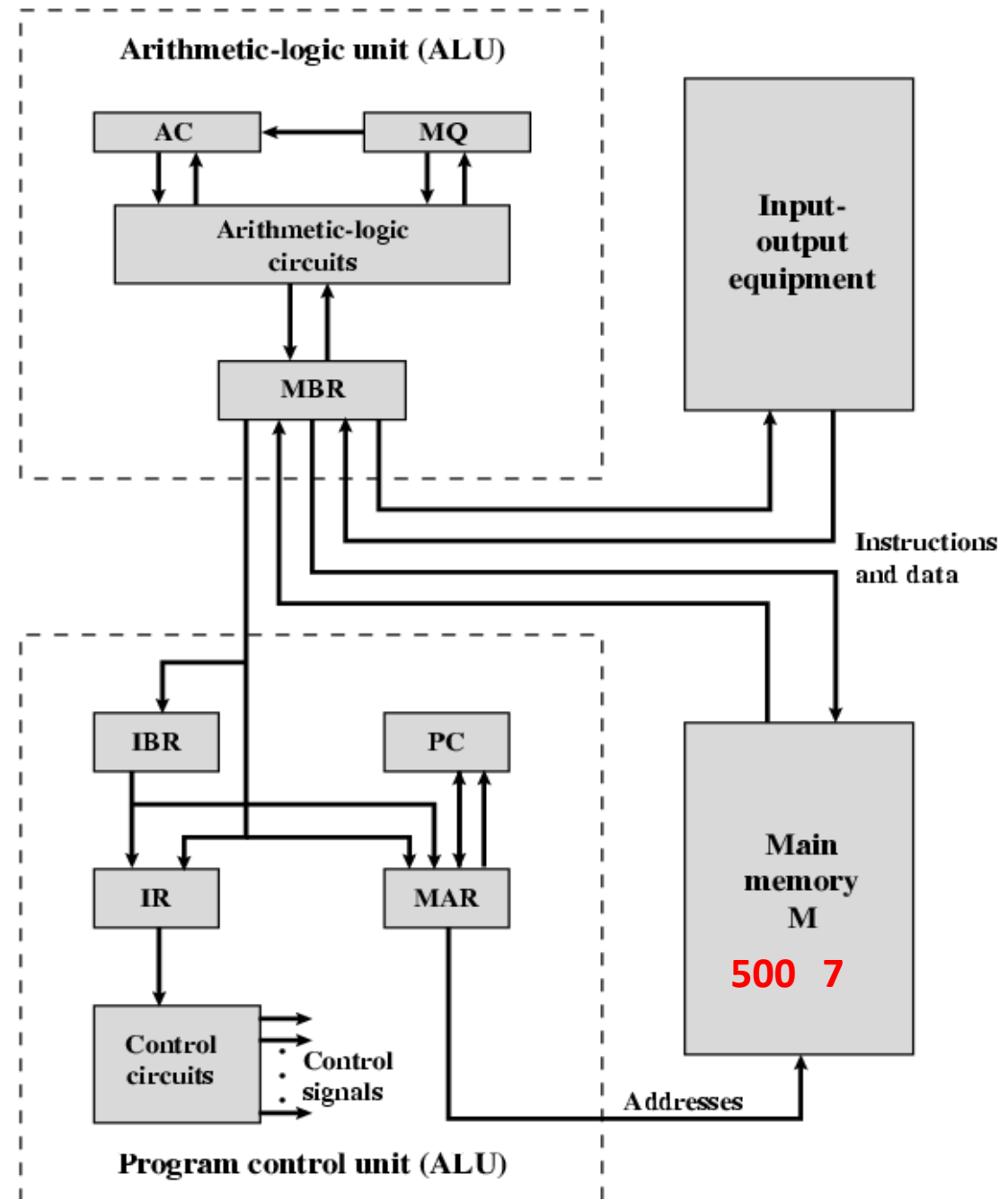
.....

500. 3

501. 4

PC	1	PC	2
MAR	500	MAR	501
MBR	3	MBR	4
IR	LOAD M(500)	IR	ADD M(501)
IBR	ADD M(501)	IBR	ADD M(501)
AC	3	AC	7

PC	2
MAR	500
MBR	7
IR	STOR M(500)
IBR	Other Ins
AC	7



IAS Architecture

Register transfer operation for addition operation

1. LOAD M(500), ADD M(501)

Register transfer operations: (PC = 1)

$MAR \leftarrow PC$

$MBR \leftarrow M[MAR]$

$IBR \leftarrow MBR[20:39]$

$IR \leftarrow MBR[0:7]$

$MAR \leftarrow MBR[8:19]$

$MBR \leftarrow M[MAR]$

$AC \leftarrow MBR$

$IR \leftarrow IBR[0:7]$

$MAR \leftarrow IBR[8:19]$

$PC \leftarrow PC + 1$

$MBR \leftarrow M[MAR]$

$AC \leftarrow AC + MBR$

2. STOR M(500), Other Instructions

▪ $MAR \leftarrow PC$

▪ $MBR \leftarrow M[MAR]$

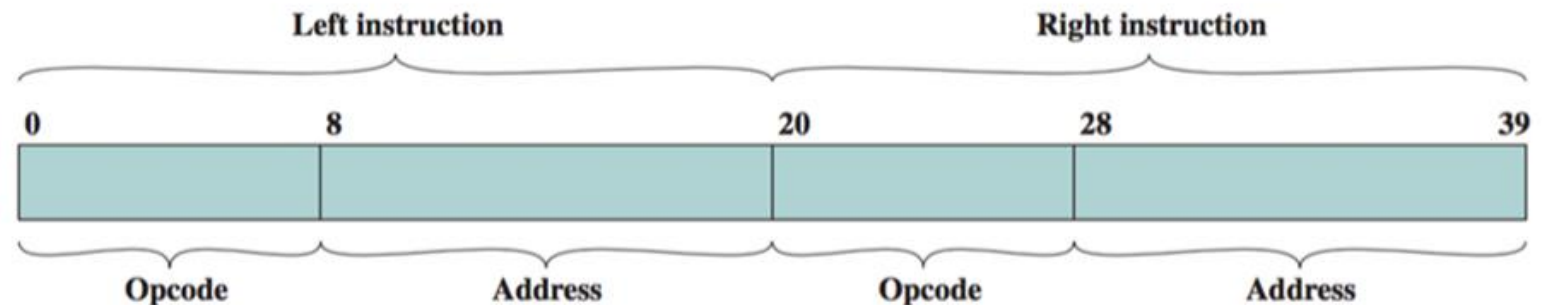
▪ $IBR \leftarrow MBR[20:39]$

▪ $IR \leftarrow MBR[0:7]$

▪ $MAR \leftarrow MBR[8:19]$

▪ $MBR \leftarrow AC$

▪ $M[MAR] \leftarrow MBR$



(b) Instruction word

IAS Instructions

The IAS computer had a **total of 21 instructions** grouped as follows:

- **Data transfer:** Move data between memory and ALU registers or between two ALU registers.
- **Unconditional branch:** Normally, the control unit executes instructions in Sequence from memory. This sequence can be changed by a branch instruction, which facilitates repetitive operations.
- **Conditional branch:** The branch can be made dependent on a condition, thus allowing decision points.
- **Arithmetic:** Operations performed by the ALU.
- **Address modify:** Permits addresses to be computed in the ALU and then inserted into instructions stored in memory.

Instruction Type	Opcode	Symbolic Representation	Description
Data transfer	00001010	LOAD MQ	Transfer contents of register MQ to the accumulator AC
	00001001	LOAD MQ,M(X)	Transfer contents of memory location X to MQ
	00100001	STOR M(X)	Transfer contents of accumulator to memory location X
	00000001	LOAD M(X)	Transfer M(X) to the accumulator
	00000010	LOAD -M(X)	Transfer -M(X) to the accumulator
	00000011	LOAD M(X)	Transfer absolute value of M(X) to the accumulator
	00000100	LOAD - M(X)	Transfer - M(X) to the accumulator
Unconditional branch	00001101	JUMP M(X,0:19)	Take next instruction from left half of M(X)
	00001110	JUMP M(X,20:39)	Take next instruction from right half of M(X)
Conditional branch	00001111	JUMP+ M(X,0:19)	If number in the accumulator is nonnegative, take next instruction from left half of M(X)
	00010000	JUMP+ M(X,20:39)	If number in the accumulator is nonnegative, take next instruction from right half of M(X)

Arithmetic	00000101	ADD M(X)	Add M(X) to AC; put the result in AC
	00000111	ADD M(X)	Add M(X) to AC; put the result in AC
	00000110	SUB M(X)	Subtract M(X) from AC; put the result in AC
	00001000	SUB M(X)	Subtract M(X) from AC; put the remainder in AC
	00001011	MUL M(X)	Multiply M(X) by MQ; put most significant bits of result in AC, put least significant bits in MQ
	00001100	DIV M(X)	Divide AC by M(X); put the quotient in MQ and the remainder in AC
	00010100	LSH	Multiply accumulator by 2, i.e., shift left one bit position
	00010101	RSH	Divide accumulator by 2, i.e., shift right one position
Address modify	00010010	STOR M(X,8:19)	Replace left address field at M(X) by 12 rightmost bits of AC
	00010011	STOR M(X,28:39)	Replace right address field at M(X) by 12 rightmost bits of AC

Problems

1. Write an appropriate assembly language code for the following operation and interpret to Von Neumann IAS architecture. (Assume that data variables 'Y' & 'Z' available at memory locations 801 & 802 respectively. And X will be stored 803 onwards.)

$$X=Y*Z$$

Solution

LOAD MQ, M(801)	$MQ \leftarrow M[801]$
MUL M(802)	$AC \leftarrow MQ * M[802]$
STOR M(803)	$M[803] \leftarrow AC$
LOAD MQ	$AC \leftarrow MQ$
STOR M(804)	$M[804] \leftarrow AC$

Note: The result of multiplying two 40-bit numbers is an 80-bit number;

40 bits are stored in the ACC and the least significant in the MQ.

They are then transferred to Memory and stored in two consecutive 40 bit word.

Problems

2. Write an ALP for the given expression using Von Neumann instruction set and write the register transfer notation for the same. Assume that B, C and D are available in the memory locations 450, 451 and 452 respectively.

$$A = (B + C) * D$$

▪ $A = (B + C) * D$

▪ Load M [450],	Add M [451]
▪ Store M [453],	Load MQ, M [453]
▪ Mul M [452],	Store M[453]
▪ Load MQ,	Store M[454]

Note: The result of multiplying two 40-bit numbers is an 80-bit number;

40 MSB bits are stored in the ACC and the remaining 'n' LSB least significant in the MQ.

They are then transferred to Memory and stored in two consecutive 40 bit word.

Problems

3. Write an ALP for the given expression using Von Neumann instruction set and write the register transfer notation for the same. Assume that A, B and X are available in the memory locations 451, 450 and 452 respectively.

$$X = A + B/2$$

▪ $X = A + B/2$

▪ Load M [450], RSH

▪ ADD M [451], Store M[452]

Problems

4. Write an ALP for the given expression using Von Neumann instruction set and write the register transfer notation for the same. Assume that B, C, D and Z are available in the memory locations 800, 801, 802 and 803 respectively.

$$Z = (B - C) / D$$

LOAD M[800]

SUB M[801]

DIV M[802]

STOR M[804]

LOAD MQ

STOR M[803]

Problems

Write an Assembly language programming for the following expressions using IAS computer Instruction set and interpret to the flow of IAS computer

1. $A = (B - C) * D$

2. $A = B * (C + D)$

3. $A = (B - C) / D$

4. $A = B / (C + D)$

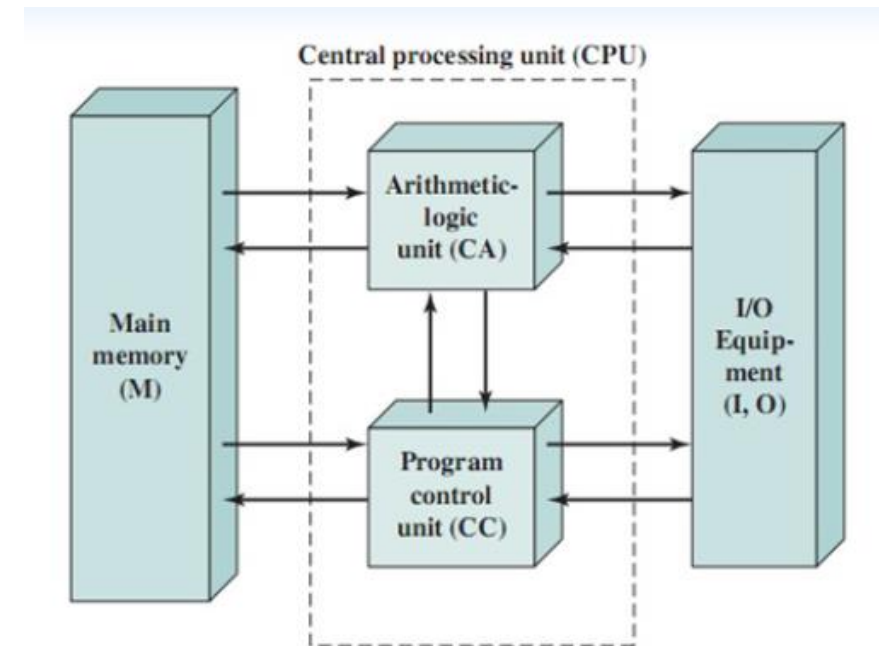
5. $A = -(B + C - D)$

6. $A = (B * 2) / 2$

Make necessary assumptions.

Disadvantages Of Von Neumann Architecture

- **Shared bus** can become a **bottleneck** if too many devices are connected to it.
- This can lead to **slow performance and reduced scalability**.
- Additionally, the **CPU can only execute one instruction at a time**, which can limit the overall speed of the system.

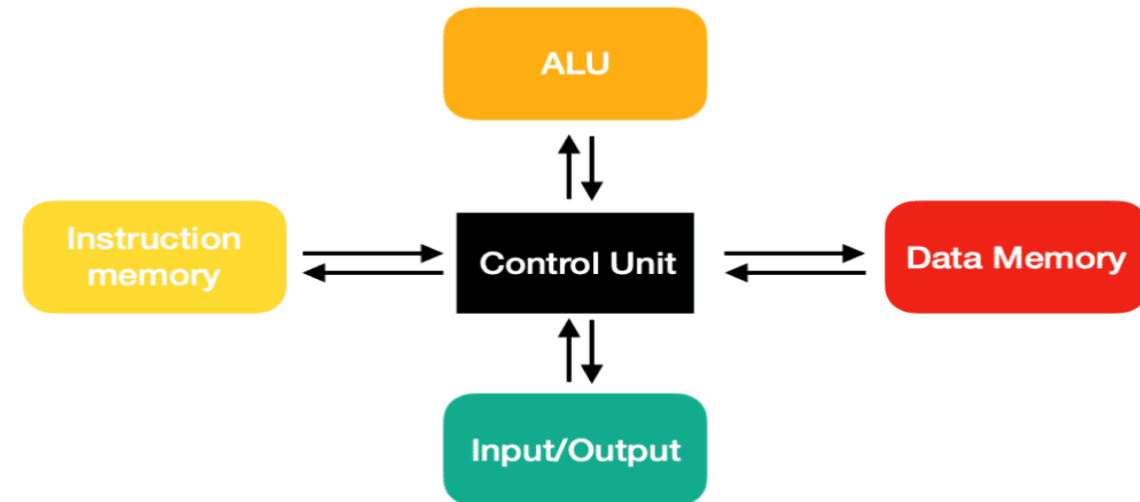


Harvard architecture

- **Von Neumann architecture**
 - instructions, and data both are stored in the same memory.
 - So same buses are used to fetch instructions and data.
 - Hence, CPU cannot do both things together (read the instruction and read/write data).
- **Harvard Architecture** - computer architecture that contains separate storage and separate buses (signal path) for instruction and data.
 - It was basically developed to overcome the bottleneck of Von Neumann's Architecture.
 - Adv: having separate buses for instruction and data - the CPU can access instructions and read/write data at the same time.

Harvard architecture - Components

- **CPU:** The central processing unit performs all the calculations and operations required to execute instructions.
- **Instruction memory:** This memory holds instructions that the CPU needs to execute. It is typically implemented as read-only memory (ROM) or flash memory.
- **Data memory:** This memory holds data that the CPU needs to perform computations. It is typically implemented as RAM.



Harvard architecture - Components

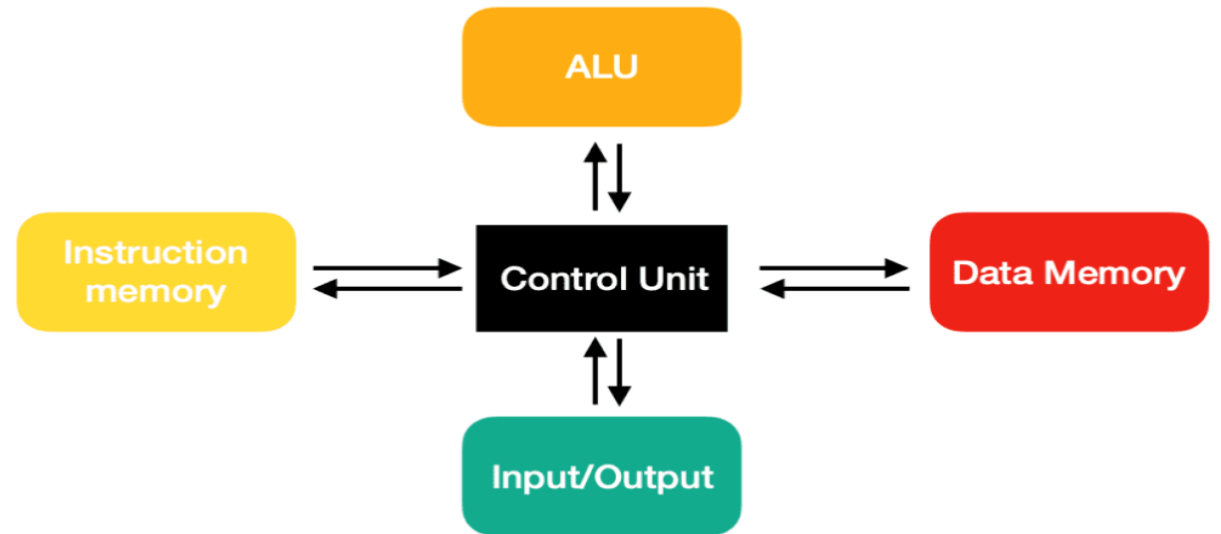
- **Input/output (I/O) devices:** used to communicate with the outside world.
- **System bus:** The system bus is a collection of wires that connect the CPU, instruction memory, data memory, and I/O devices - used to transmit data, instructions, and control signals between these components.

Advantage of Harvard Architecture:

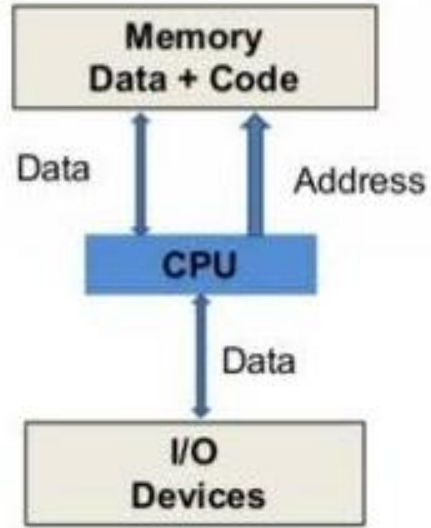
- Fast and efficient data access
- Better performance
- Suitable for real-time applications
- Security

Disadvantages of Harvard Architecture:

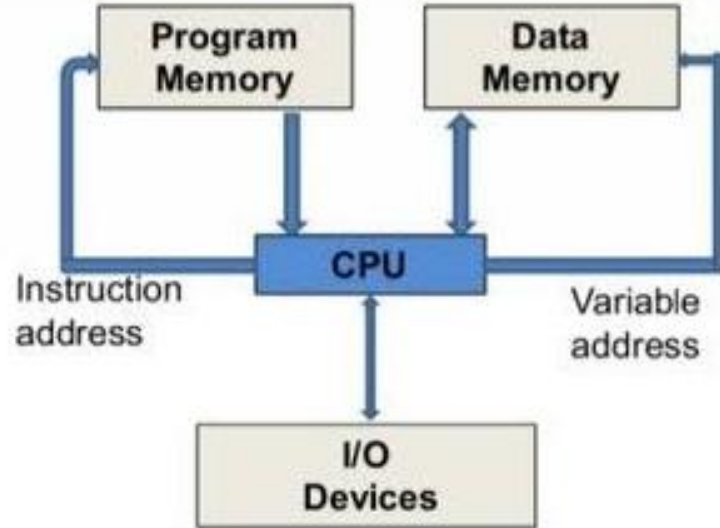
- Complexity
- Limited flexibility
- Higher memory requirements



Von Neumann Vs Harvard architecture



Von Neumann Machine



Harvard Machine

Von-Neumann (Princeton architecture)	Harvard architecture
www.eemadeeasy.com The diagram shows a CPU box on the left and a larger box on the right containing 'Program Memory' and 'Data Memory'. A single horizontal bus line connects the CPU to both memory blocks. A double-headed arrow labeled 'Data' is positioned above the bus line, and a single-headed arrow labeled 'Address Bus' points from the CPU to the memory blocks below the bus line.	The diagram shows a CPU box on the left and two separate boxes on the right: 'Data Memory' (top) and 'Program Memory' (bottom). For the Data Memory, there is a double-headed arrow labeled 'Data' and a single-headed arrow labeled 'Address Bus' pointing from the CPU. For the Program Memory, there is a double-headed arrow labeled 'Data' and a single-headed arrow labeled 'Address Bus' pointing from the CPU.

Von Neumann Vs Harvard architecture

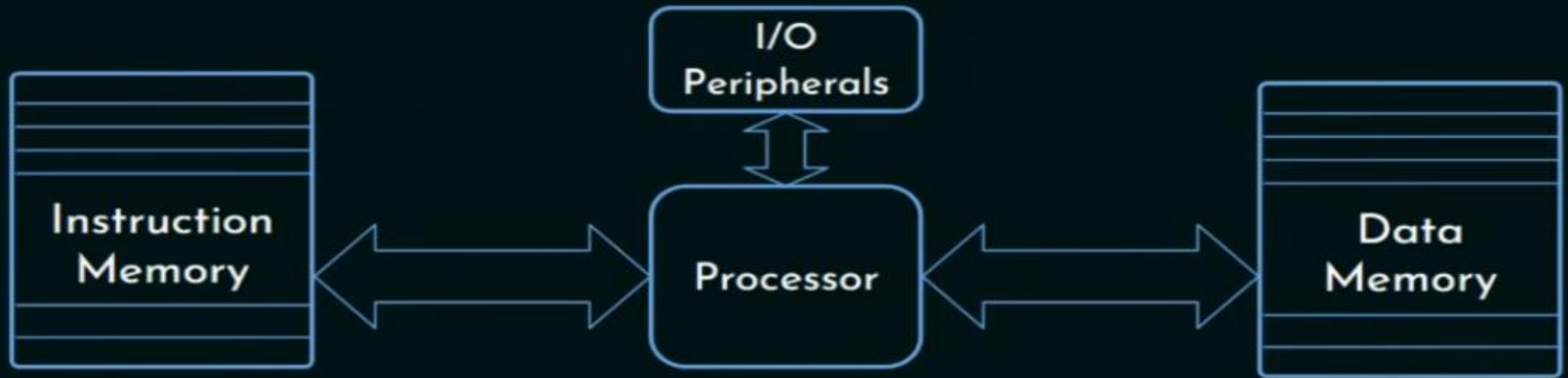
Aspect	Von Neumann Architecture	Harvard Architecture
Memory	Single memory for instructions and data	Separate memory for instructions and data
Access	CPU accesses instructions and data through a shared bus	CPU accesses instruction and data memory spaces separately
Performance	Can become a bottleneck if too many devices are connected to the shared bus	Improved performance because the CPU can access both instruction and data memory simultaneously
Modification	Easier to modify programs as instructions and data are stored in the same memory	More difficult to modify programs that require the CPU to modify its own code
Data sharing	Easy to share data between different parts of a program	More difficult to share data between different parts of a program
Applications	Suitable for general-purpose computing where flexibility is required	Suitable for embedded systems where performance is critical and code is not frequently modified

Modified Harvard Architecture

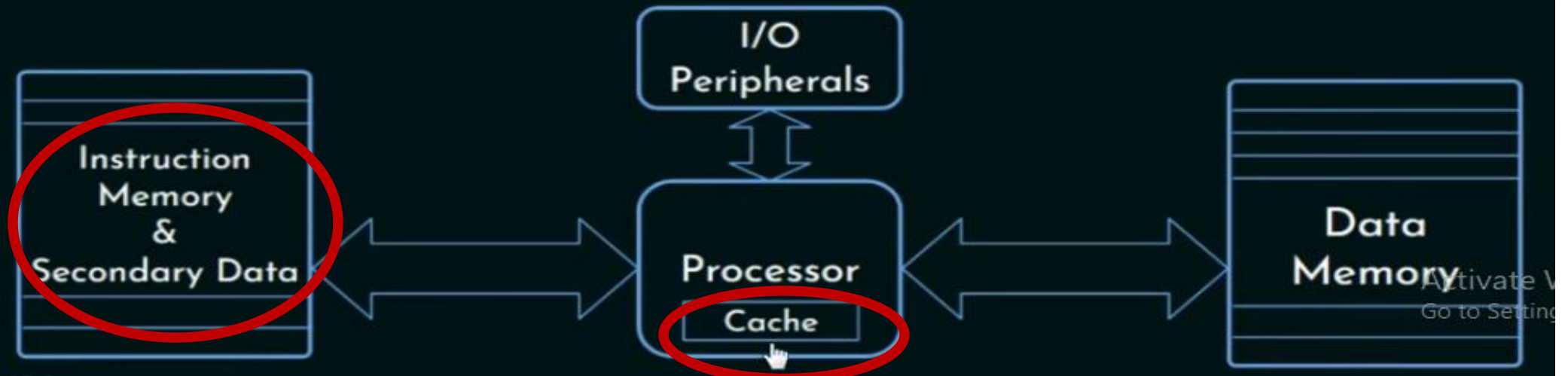
- Very similar to Harvard architecture machine - CPU concurrent access to two (or more) memory buses
- But
 - Separate Data cache & Instruction cache – placed in CPU
 - No strict separation between instruction and data memory (Can be used combinedly)
- While the CPU executes from **cache**, it acts as a **pure Harvard machine**.
- When accessing backing **memory**, it acts like a **von Neumann machine**.

Harvard Vs Modified Harvard Architecture

- **Harvard Architecture :**



- **Modified Harvard Architecture :**



RISC and CISC Architecture

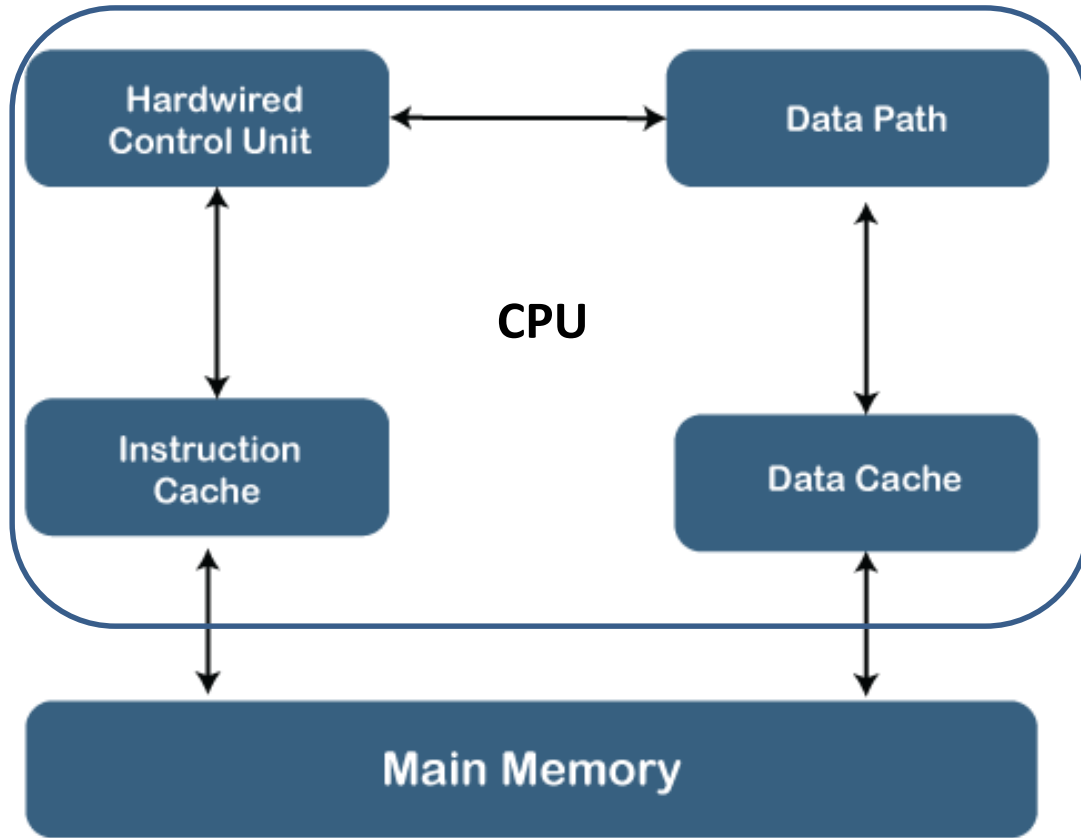
RISC

- RISC stands for **Reduced Instruction Set Computer** Processor
- is a microprocessor architecture with a **simple collection and highly customized set of instructions.**
- It is built **to minimize the instruction execution time** by optimizing and limiting the number of instructions.

Characteristics

- It has **simpler instructions** and thus simple instruction decoding.
- More general-purpose registers.
- The instruction takes **one clock cycle** in order to get executed.
- The instruction comes under the size of a **single word.**
- **Pipeline** can be easily achieved.
- Few data types.
- **Simpler addressing modes**

RISC Architecture



- **Hardware of RISC architecture** is designed to execute the **instruction quickly** - because of the more precise and smaller number of instructions and a large number of registers.
- **Data path** - store and manipulate data in a computer.
 - responsible for **managing data within the processor** and its movement **between the processor and the memory**.
- **Cache** - to reduce the access time to the main memory.
 - **Instruction cache** - for retrieving and storing the data of frequently used instructions - speeds up the process of instruction execution.
 - **Data cache** - provides storage for frequently used data from the main memory.

RISC and CISC Architecture

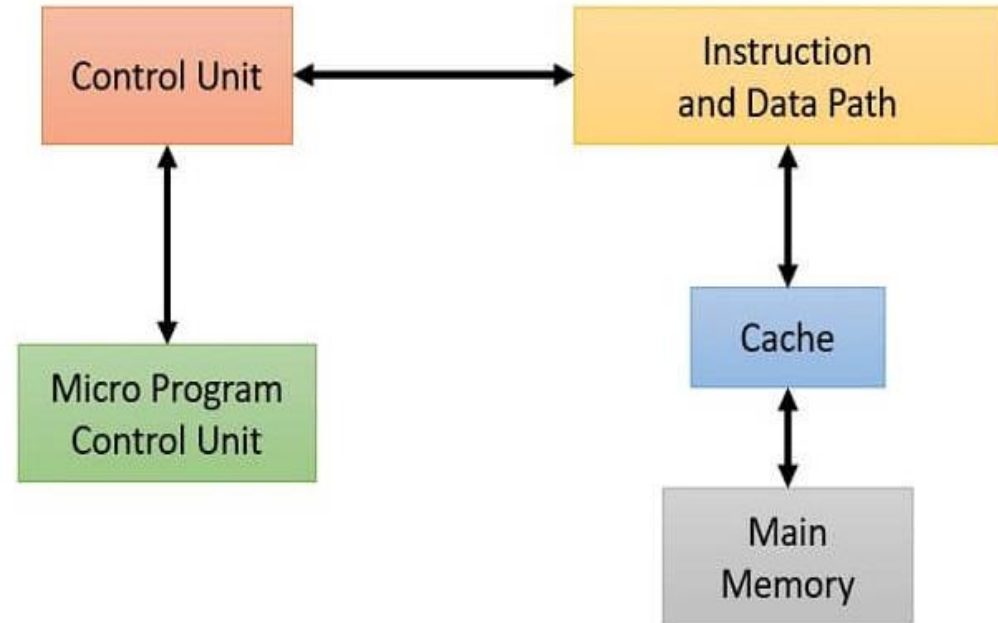
CISC

- CISC Stands for **Complex Instruction Set Computer**, developed by the Intel.
- has a large collection of **complex instructions** that range from **simple to very complex**.
- **specialized in the assembly language level**, which takes a **long time to execute** the instructions.

Characteristics

- Instructions are **complex**, and thus it has complex instruction decoding.
- Instructions may take **more than one clock cycle** in order to get executed.
- Instruction is **larger than one-word size**.
- **Lesser general-purpose registers** since the operations get performed only in the memory.
- More data types.
- **Complex addressing modes**.

CISC Architecture



- **Cache memory** - for holding both **data and instructions**.
 - Thus, they **share the same path for both** instructions and data.
- **Instructions with variable length format** - Thus, the number of **clock cycles** required to execute the instructions may be varied.
- **Microprogram control unit** - uses a series of **microinstructions** of the microprogram stored in the “**control memory**” of the **microprogram control unit** and **generate the control signals**.
- **Control units** - **access the control signals** produced by the microprogram control unit & operate the functioning of processors hardware.

RISC and CISC Architecture

Example:

$$A = B + C$$

RISC instruction

Load R2, B

Load R3, C

Add R4, R2, R3

Store R4, A

CISC instruction

Move A, B

Add A, C

RISC and CISC Architecture

RISC	CISC
1. RISC stands for Reduced Instruction Set Computer.	1. CISC stands for Complex Instruction Set Computer.
2. RISC processors have simple instructions taking about one clock cycle. The average clock cycle per instruction (CPI) is 1.5	2. CSIC processor has complex instructions that take up multiple clocks for execution. The average clock cycle per instruction (CPI) is in the range of 2 and 15.
3. Performance is optimized with more focus on software	3. Performance is optimized with more focus on hardware.
4. It has no memory unit and uses a separate hardware to implement instructions..	4. It has a memory unit to implement complex instructions.
5. It has a hard-wired unit of programming.	5. It has a microprogramming unit.
6. The instruction set is reduced i.e. it has only a few instructions in the instruction set. Many of these instructions are very primitive.	6. The instruction set has a variety of different instructions that can be used for complex operations.
7. Complex addressing modes are synthesized using the software.	7. CISC already supports complex addressing modes

RISC and CISC Architecture

8. Multiple register sets are present	8. Only has a single register set
9. RISC processors are highly pipelined	9. They are normally not pipelined or less pipelined
10. The complexity of RISC lies with the compiler that executes the program	10. The complexity lies in the microprogram
11. Execution time is very less	11. Execution time is very high
12. Code expansion can be a problem	12. Code expansion is not a problem
13. Decoding of instructions is simple.	13. Decoding of instructions is complex
14. It does not require external memory for calculations	14. It requires external memory for calculations
15. The most common RISC microprocessors are Alpha, ARC, ARM, AVR, MIPS, PA-RISC, PIC, Power Architecture, and SPARC.	15. Examples of CISC processors are the System/360, VAX, PDP-11, Motorola 68000 family, AMD and Intel x86 CPUs.
16. RISC architecture is used in high-end applications such as video processing, telecommunications and image processing.	16. CISC architecture is used in low-end applications such as security systems, home automation, etc.

End of Module 1

Module:1	Introduction To Computer Architecture and Organization	5 Hours
Overview of Organization and Architecture –Functional components of a computer: Registers and register files - Interconnection of components - Overview of IAS computer function - Organization of the von Neumann machine - Harvard architecture - CISC & RISC Architectures.		